Global digital University

20/10/26



TaskFlow

Application de gestion de projet

Formation : Concepteur développeur d'applications Javascript

Présenté par : Elia SOINARD

SOMMAIRE

Table des matières

I. Présentation 4
II. Versionning 5
III. Backend 7
A. Analyse du besoin 7
B. Création du serveur 8
C. Base de données 9
D. Fichiers d'environnement 10
E. Structure de l'application 11
1. Arborescence 11
A. Middleware 11
B. Models 12
C. Controllers 15
D. Routes 19
E. Application 20
2. Couches logicielles 21
3. Parcours d'une requête 22
F. Sécurité 23
1. JWT 23
2. Bcrypt 24
3. Authentification 25
4. Validation des données 25
5. Routes 26
6. Contrôle d'accès 27
7. Cors/rate limiter 27
G. Tests 28
1. Postman 28
2. Jest/Supertest 29
H. Documentation 31
1. Fichier README.md 31
2. Swagger 32
IV. Frontend 34
A. Analyse du besoin 34

B. Choix techniques 35
C. Fichiers d'environnement 35
D. Structure de l'application 36
1. Arborescence 36
2. Pages et composants 37
3. Navigation entre les pages 38
4. Communication avec l'API 38
5. Gestion de la connexion 39
6. Détail des pages et composants 40
A. Components - Project 40
B. Components – menu et déconnexion 40
C. Formulaire - CreateProject 41
D. Formulaire - EditProject 42
E. Page Tasks 43
7. Couches logicielles 44
8. Parcours d'une action utilisateur 45
E. Interface utilisateur 45
F. Sécurité 46
G. Tests 47
H. Documentation 48
V. Docker 49
A. Dockerfile du backend 49
B. Dockerfile du frontend 50
C. Docker Compose 51
D. Fichier .dockerignore 51
VI. DevOps 52
A. Qualité du code 53
B. Pipeline CI/CD avec GitHub Actions 54
C. Sécurisation du déploiement 55
D. Hébergement et mise en production 56
VII. Logiciels utilisés 57

I. Présentation

L'objectif de ce rapport est de vous présenter la création de l'application TaskFlow utilisée pour la gestion de projet.

Cette application gère la création de projet, la gestion de ceux-ci ainsi que les tâches et tags associés.
J'ai pu faire le développement complet de l'application, de l'analyse du besoin jusqu'à son déploiement.

L'application doit respecter ces spécificités.

Utilisateurs (authentifiés par JWT) :
- Inscription
- Login
- Récupération du profil

Projets :
- Seul le créateur peut créer/supprimer des tâches et des projets
- Ajout/suppression de membres par le créateur ou un rôle manager du projet
- Seul le créateur d'un projet peut créer/supprimer ses tâches

Tâches :
- Création (par créateur du projet)
- Mise à jour (créateur ou assigné selon champ)
- Ouvrir/fermer (l'assigné peut clore/réouvrir)
- Suppression (créateur)
- L'assigné·e d'une tâche peut passer l'état à closed et le rouvrir

Autorisations :
- Un utilisateur ne voit que les projets dont il est membre (sauf ROLE_MANAGER)
- Le créateur gère les membres et les tâches du projet
- L'assigné·e d'une tâche peut changer son état (open/closed)

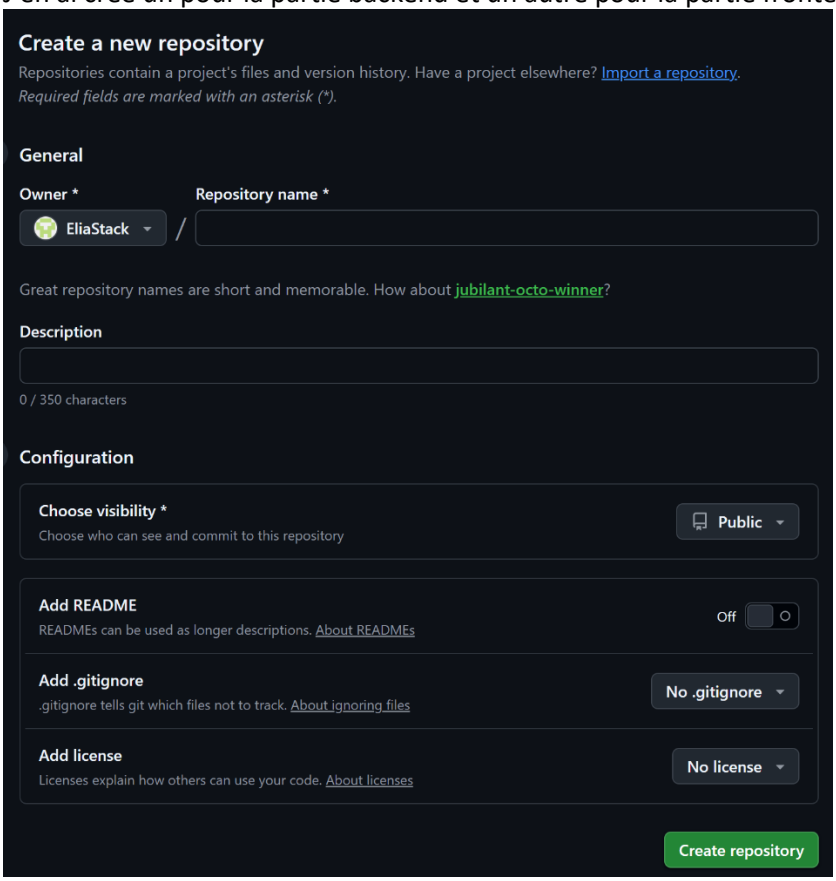
Sécurité :
- Un utilisateur non membre d'un projet ne peut ni lire ni modifier ses tâches (403)
- Les entrées sont validées (formats, longueurs, cohérence des dates)

Base de données :
- MongoDB Atlas

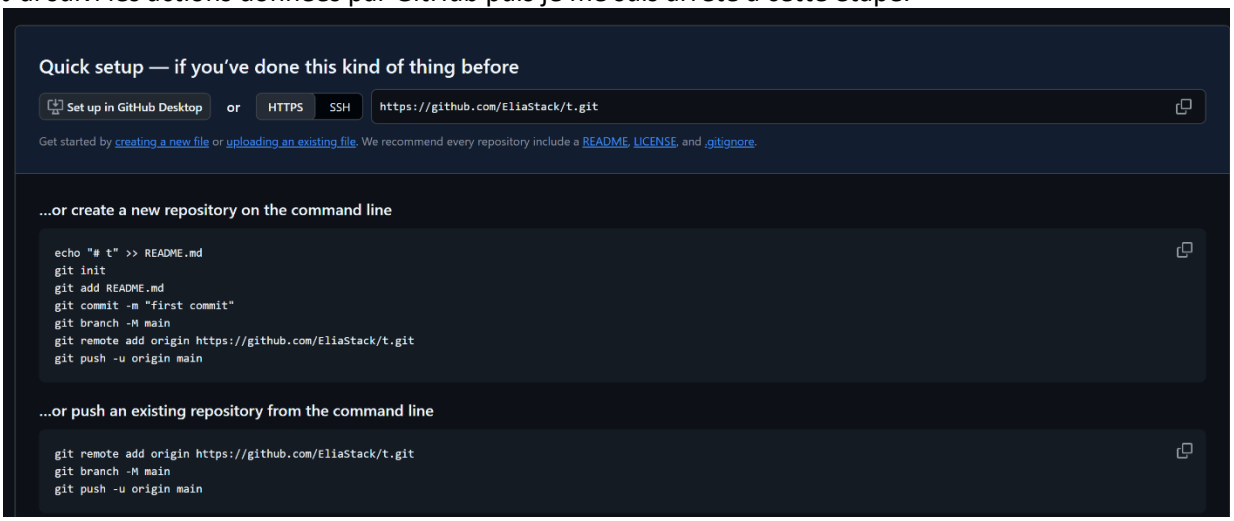
L'application est développée sur l'IDE VS Code.
Pour la base de données, j'ai utilisé MongoDB, l'API est une API Rest.
Pour le backend j'ai utilisé Node.js et pour le frontend React.

II. Versionning

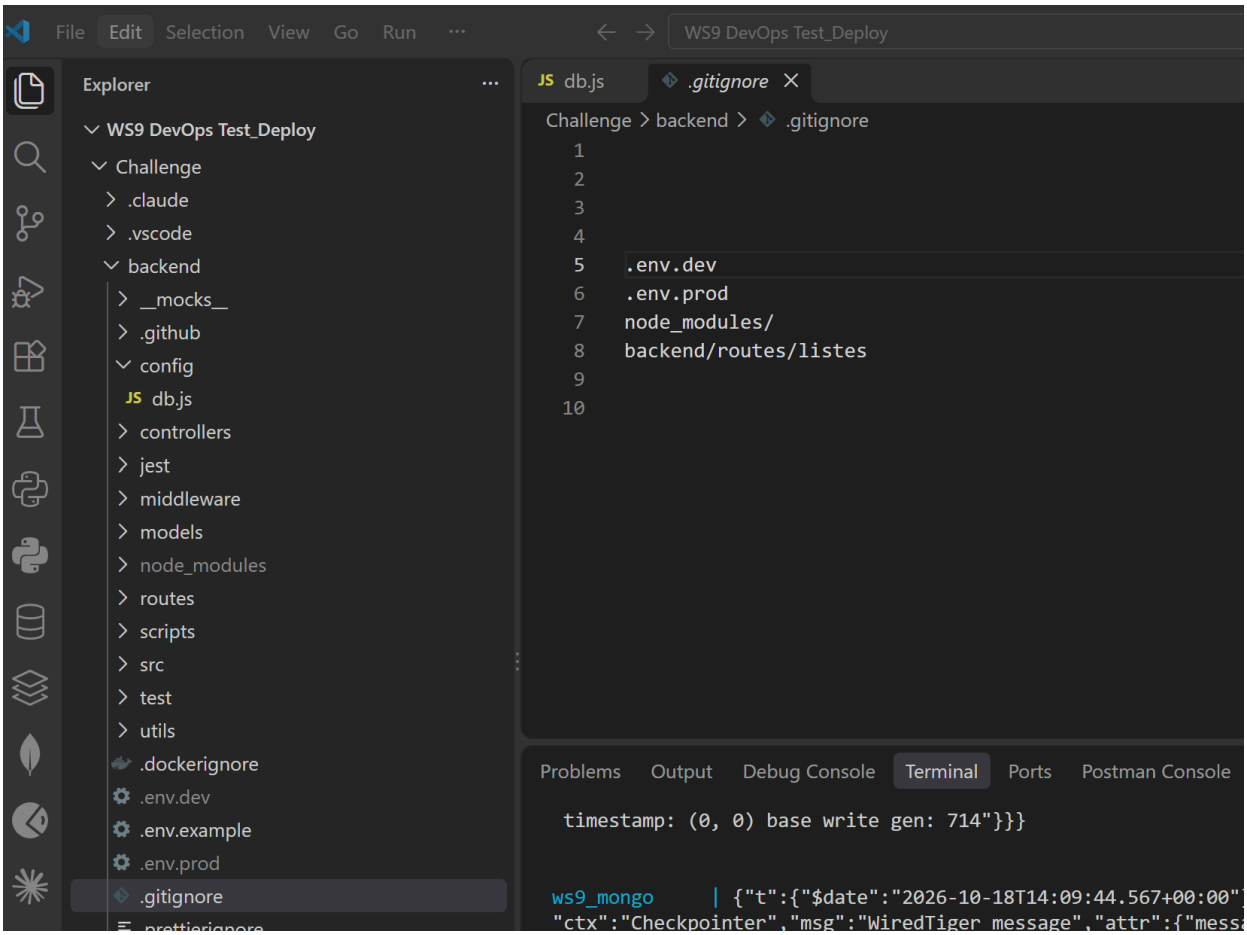
J'ai commencé par créer les dépôts GitHub que j'utilise pour le versionning des projets.
J'en ai créé un pour la partie backend et un autre pour la partie frontend.



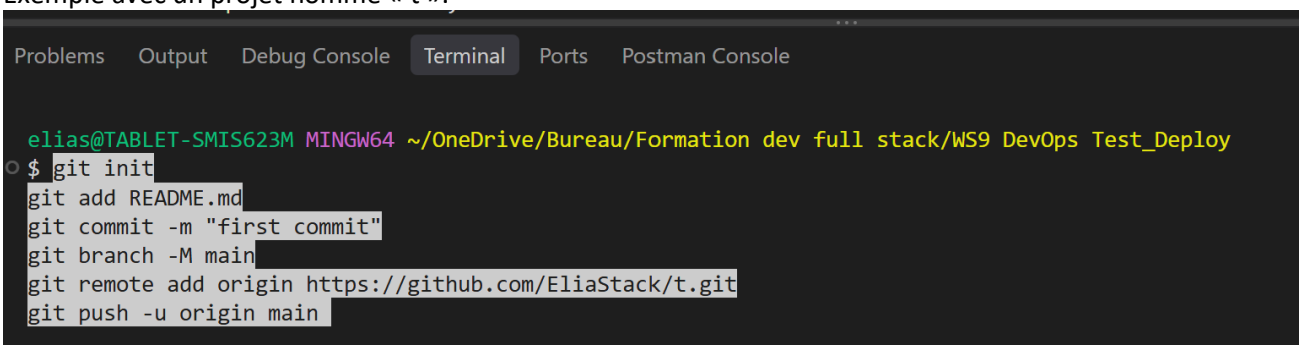
J'ai suivi les actions données par GitHub puis je me suis arrêté à cette étape.



Je rajoute le fichier .gitignore qui va me permettre d'ignorer les fichiers mis dedans quand je fais des commits, et d'éviter de rendre publics des fichiers sensibles comme le .env.



J'ai fait le lien entre le répertoire GitHub et mon projet dans un terminal Bash en rentrant les commandes données par GitHub.
Exemple avec un projet nommé « t ».

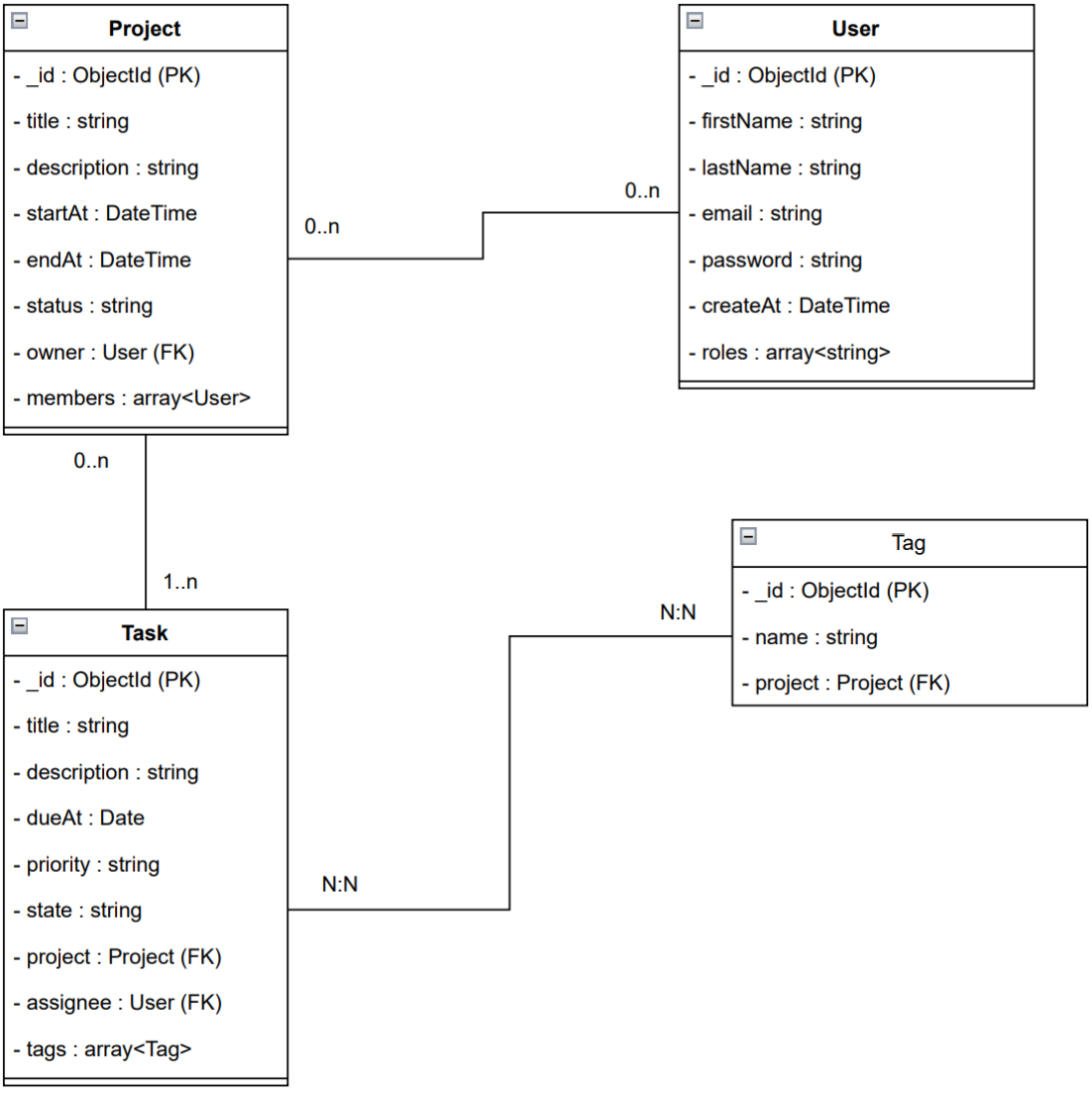


III. Backend

A. Analyse du besoin

J'ai créé un diagramme de classe UML (Unified Modeling Language) qui m'a permis de représenter graphiquement les besoins et me faire une première idée du logiciel que je veux créer.

J'ai utilisé les FK et PK qui seront ensuite modifiées pour s'adapter au format Mongoose.



B. Création du serveur

J'ai créé un serveur pour lancer l'API séparément pour pouvoir tester l'API sans démarrer le serveur.
Le serveur récupère le port et vérifie s'il est valide.
Il est ensuite affecté à l'application.
Le serveur gère les erreurs de démarrage :

- EADDRINUSE, le port est déjà pris
- EACCES, je n'ai pas accès

Ensuite le serveur écoute

```
console.log('Création du serveur');

const dns = require('node:dns');
dns.setServers(['8.8.8.8', '1.1.1.1']);

//Création du serveur
const http = require('http'); //importer le package http qui donne accès à http
const app = require('./app'); //Lien JS de l'appli

const normalizePort = (val) => {
    //normalizePort=renvoie un port valide, qu'il soit fourni sous la forme d'un numéro ou d'une chaine
    const port = parseInt(val, 10);

    if (isNaN(port)) {
        return val;
    }
    if (port >= 0) {
        return port;
    }
    return false;
};
const port = normalizePort(process.env.PORT || '3000');
app.set('port', port);
const errorHandler = (error) => {
    //errorHandler=recherche les différentes erreurs et les gère de manière appropriée. Elle est ensu
    if (error.syscall !== 'listen') {
        throw error;
    }
    const address = server.address();
    const bind =
        typeof address === 'string' ? 'pipe ' + address : 'port: ' + port;
    switch (error.code) {
        case 'EACCES':
            console.error(bind + ' requires elevated privileges.');
            process.exit(1);
            break;
        case 'EADDRINUSE':
            console.error(bind + ' is already in use.');
            process.exit(1);
            break;
        default:
            throw error;
    }
};

const server = http.createServer(app); //Lien enter le serveur et l'application

server.on('error', errorHandler);
server.on('listening', () => {
    const address = server.address();
    const bind =
        typeof address === 'string' ? 'pipe ' + address : 'port ' + port;
    console.log('Listening on ' + bind);
});

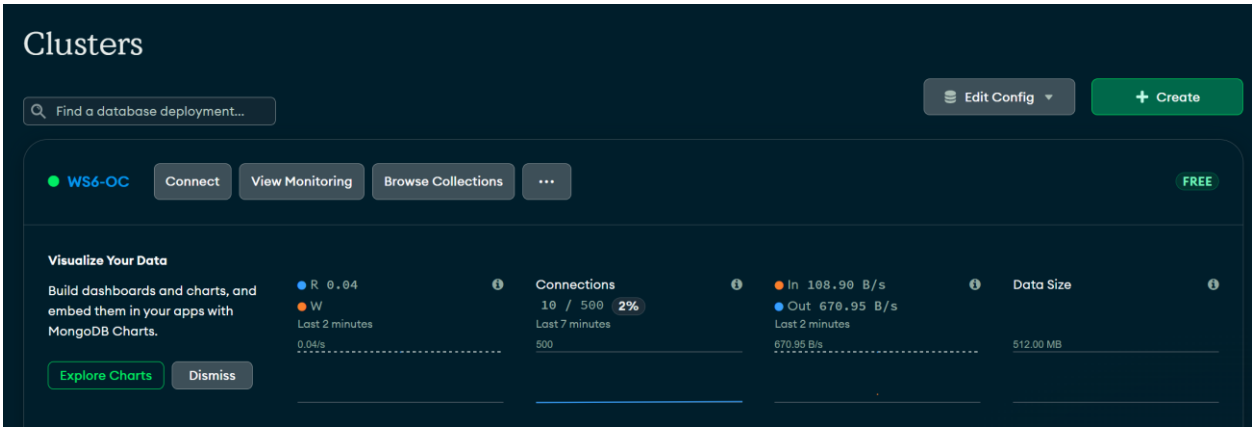
//Attente des requêtes envoyées en écoutant le port
server.listen(port);
```

C. Base de données

J'ai installé Node.js et la librairie mongoose.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE
elias@TABLET-SMIS623M MINGW64 ~/OneDrive/Bureau/Formation dev full stack/WS6 Créer une applicat
s et MongoDB/Challenge/backend
$ npm install mongoose
25 packages are looking for funding
  run `npm fund` for details
```

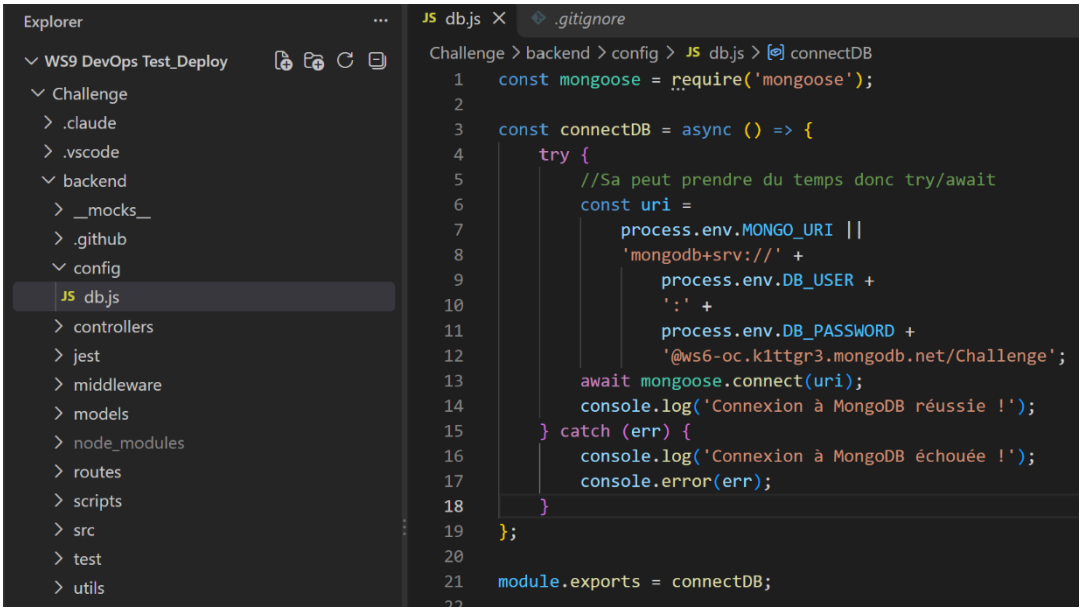
Je suis allée sur MongoDB Atlas pour créer mon cluster.



J'ai fait le lien avec le backend en créant un fichier db.js avec une fonction appelée connectDB que j'ai mis dans un dossier à part que j'ai nommé config. Cette fonction sera appelée plus tard dans le fichier app.js.

Elle me permet de me connecter à ma base de données MongoDB, j'ai mis les informations user et password dans le fichier .env.

Le fichier .env étant listé dans le .gitignore, il ne sera jamais envoyé sur GitHub.



A ce stade la base est créée mais ne contient pas encore de données.

D. Fichiers d'environnement

J'ai différents fichiers qui me servent à contenir les variables d'environnement ainsi que les données sensibles qui doivent rester secrètes :

- .env.dev contient les valeurs pour développer sur le pc
- .env.prod contient les valeurs pour la production
- .env.example est un modèle avec les noms de variable mais sans les vraies valeurs pour que quelqu'un qui clone le projet sache quoi remplir

Chaque fichier contient ces variables :

- JWT_SECRET : la clé qui signe les tokens de connexion
- DB_USER / DB_PASSWORD : les identifiants pour se connecter à MongoDB Atlas
- URL_Front : l'adresse du frontend, utilisée par CORS pour accepter ses requêtes

La sélection des bonnes variables se fait dans le fichier app.js

```
dotenv.config({ path: `.env.${process.env.NODE_ENV || 'dev'}` });
```

En fonction des scripts lancés dans le terminal ou au moment du push de GitHub Action

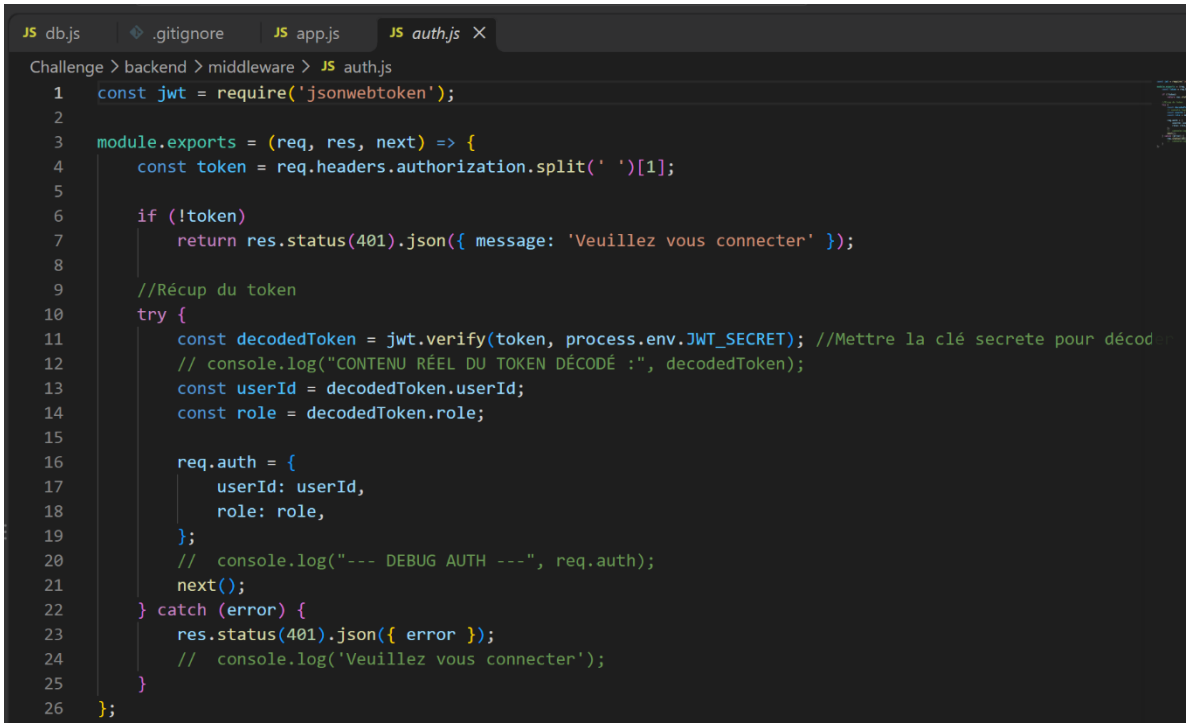
- npm run dev → NODE_ENV=dev → charge **.env.dev**
- npm start → NODE_ENV=prod → charge **.env.prod**

E. Structure de l'application

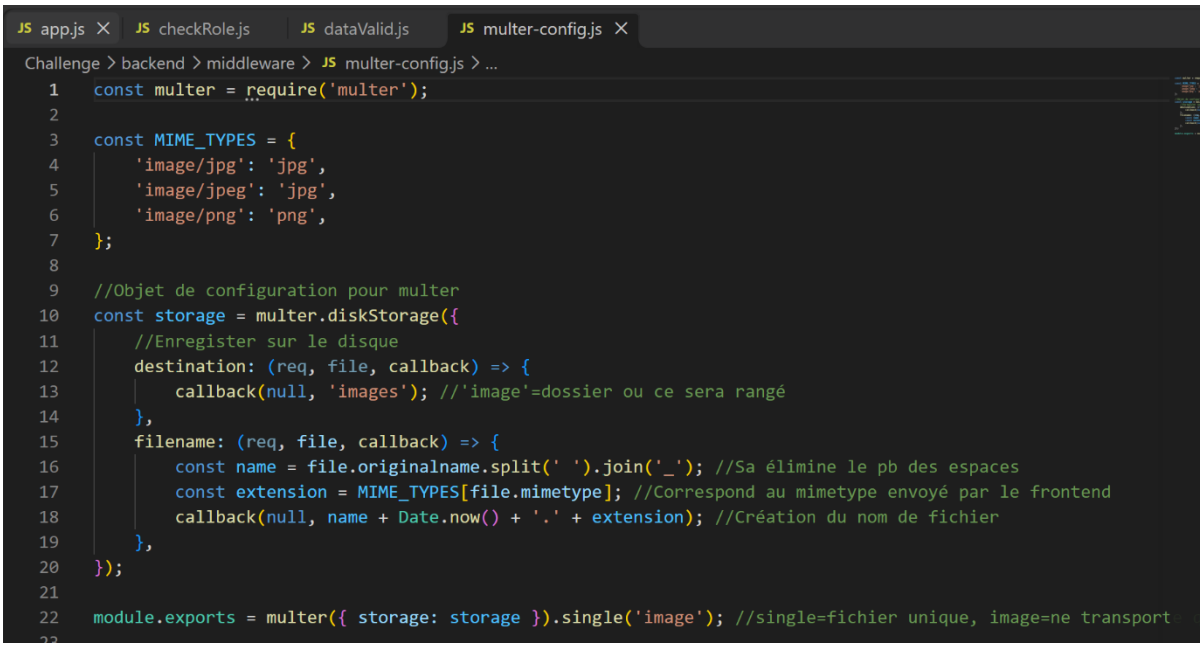
1. Arborescence

A. Middleware

J'ai créé quatre middlewares pour gérer l'authentification, les fichiers images et données.
Le middleware auth.js concerne les tokens. Si je n'ai pas de token j'ai une erreur 401 et je dois me connecter.
En me connectant avec les bons identifiants, le token est vérifié et décodé avec la clef secrète puis stocké sinon j'ai une erreur 401 et je dois me connecter.



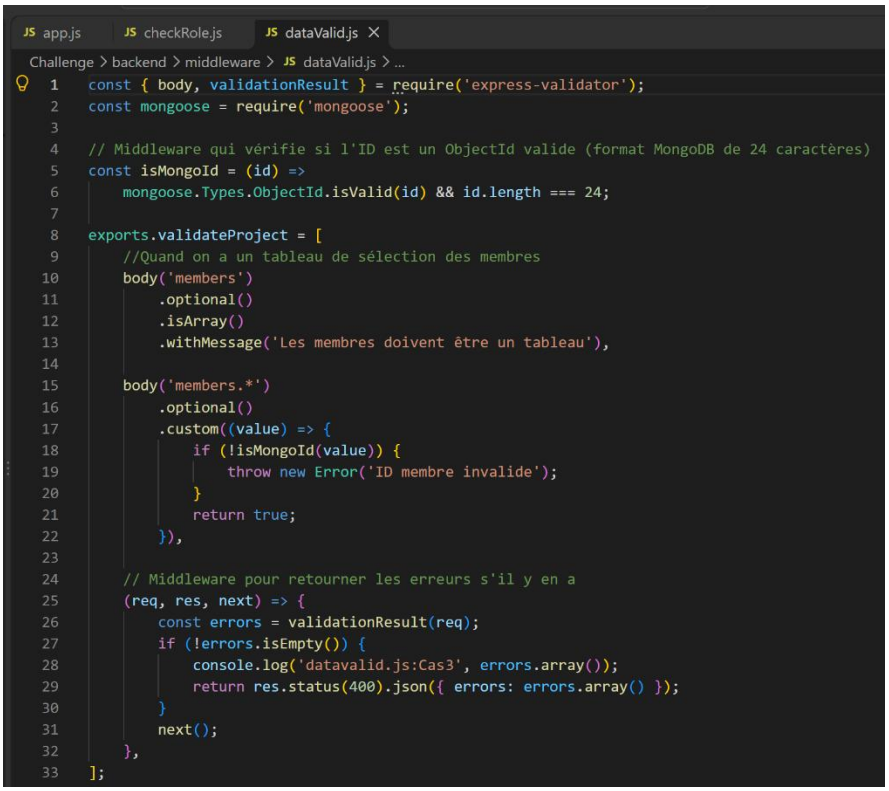
Le middleware multer-config.js me permet de référencer les différents formats d'image, de les stocker en mettant le nom en enlevant les espaces avec la date puis d'ajouter le type d'extension à la fin.



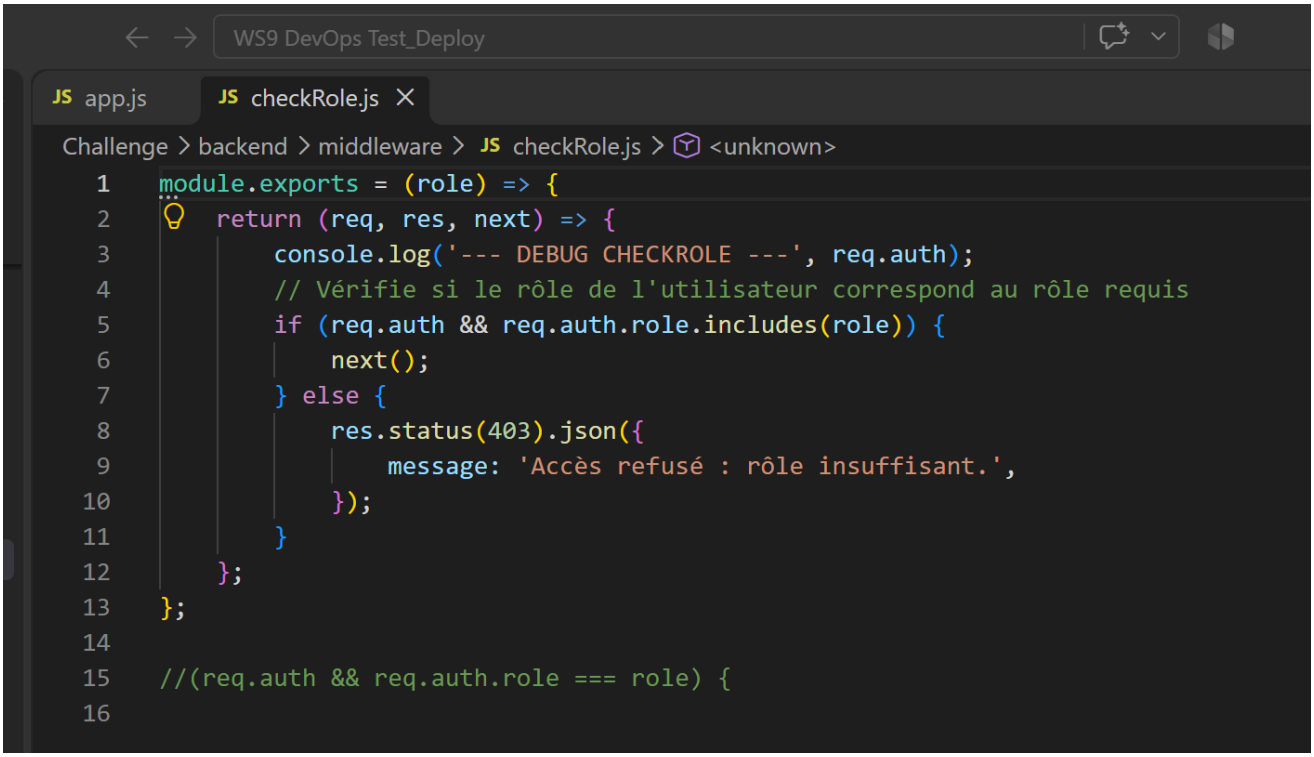
Le middleware dataValid.js vérifie que :

- l'ID est au format Mongoose donc 24 caractères
- Les membres fassent partie d'un tableau
- Les membres ont tous un format valide

S'il y a une erreur, le middleware me retourne l'erreur 400.

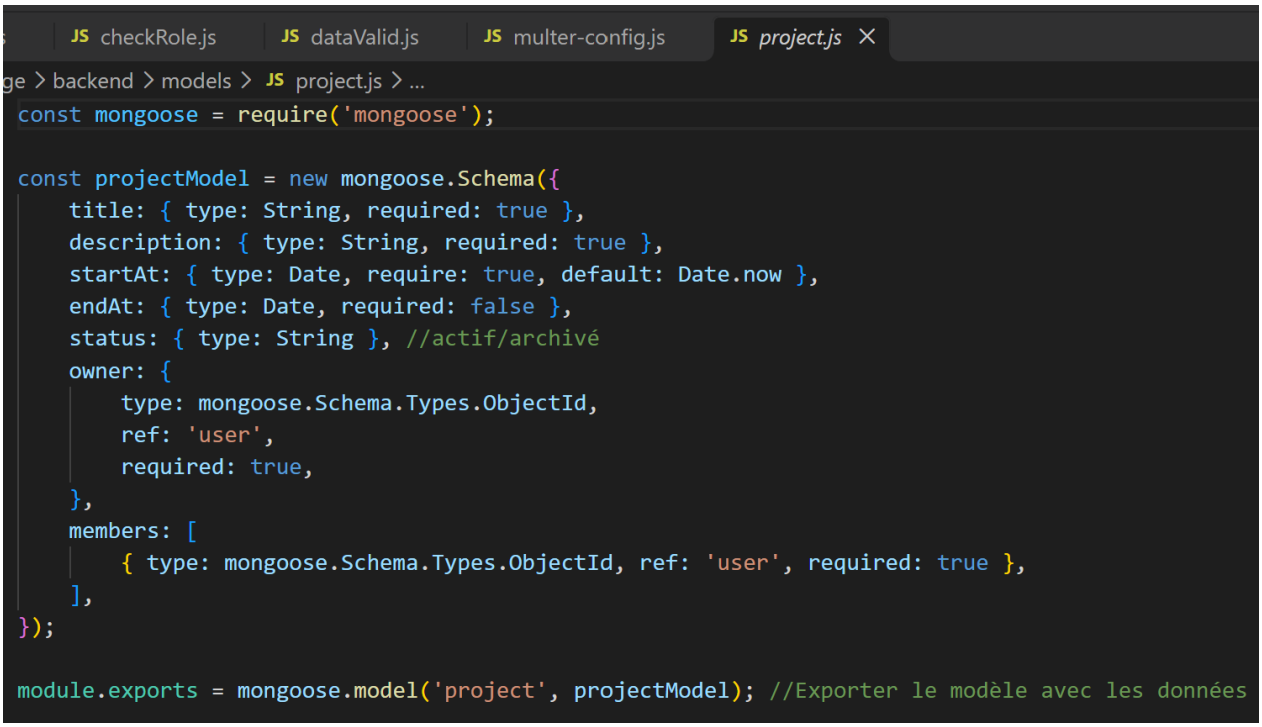


Le middleware checkRole.js vérifie que le rôle de l'utilisateur connecté correspond à celui paramétré dans la route sinon il me retourne l'erreur 403.

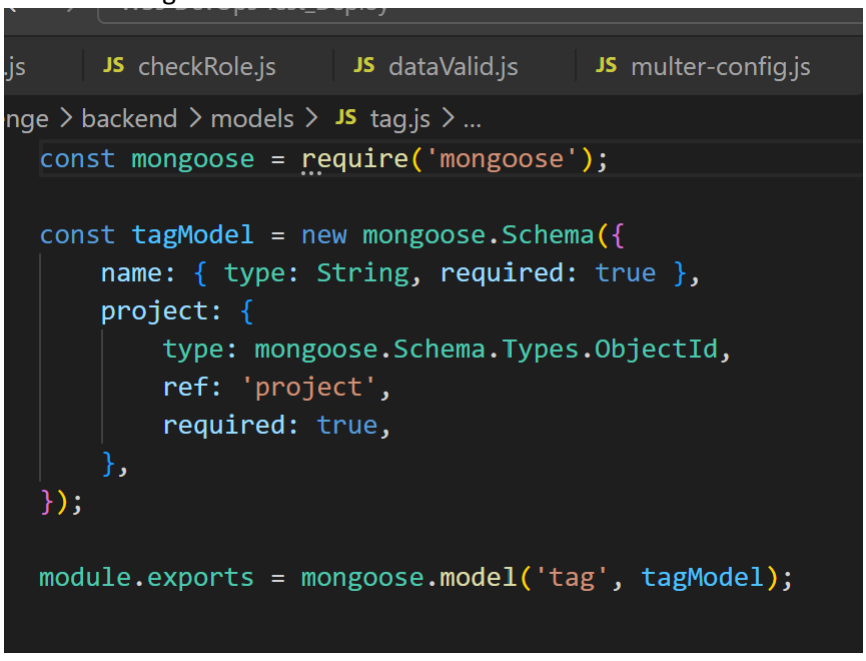


J'ai repris mon diagramme de classe pour créer mes modèles.
Pour ce projet j'en ai créé quatre au format Mongoose.
Pour chacun d'entre eux je commence par créer une constante qui comportera le modèle puis j'utilise la fonction « new mongoose.Schema» dans laquelle je mets ma structure de donnée.
Quand mon modèle est terminé je l'exporte en utilisant « module.exports ».

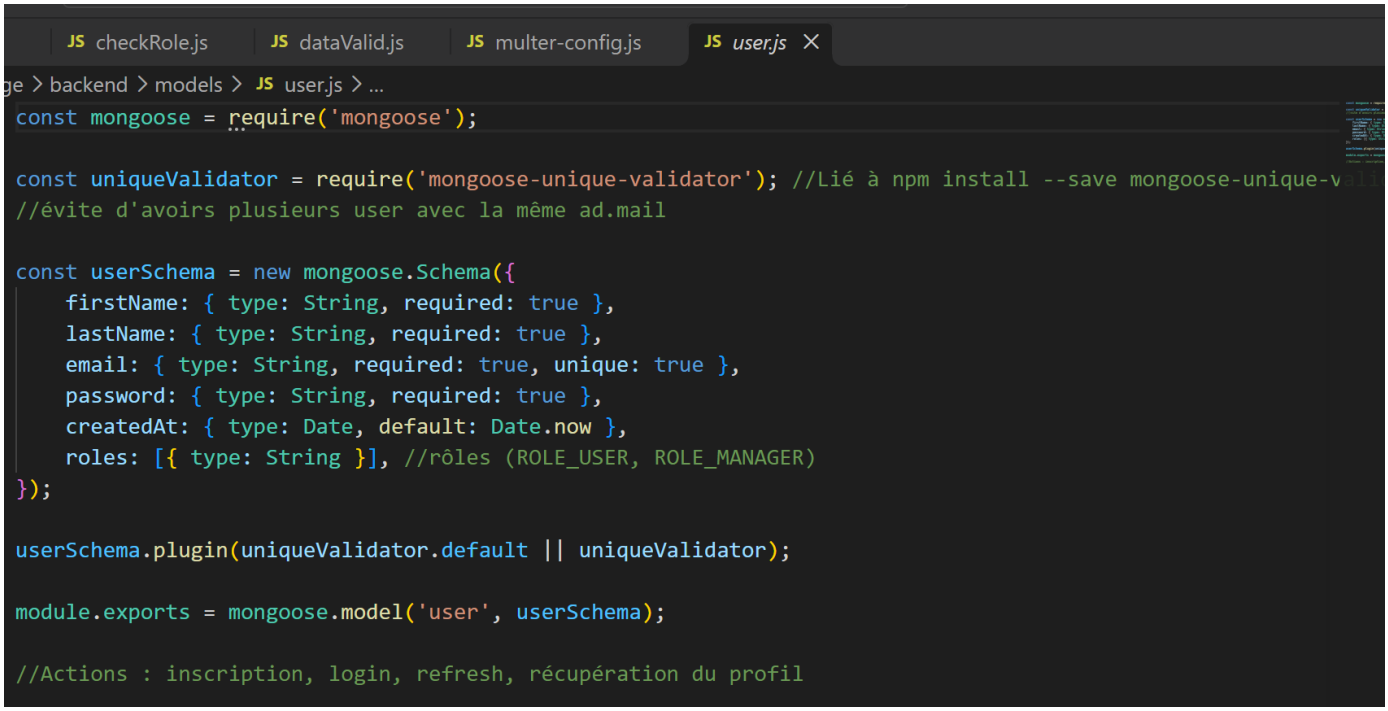
Modèle de project :



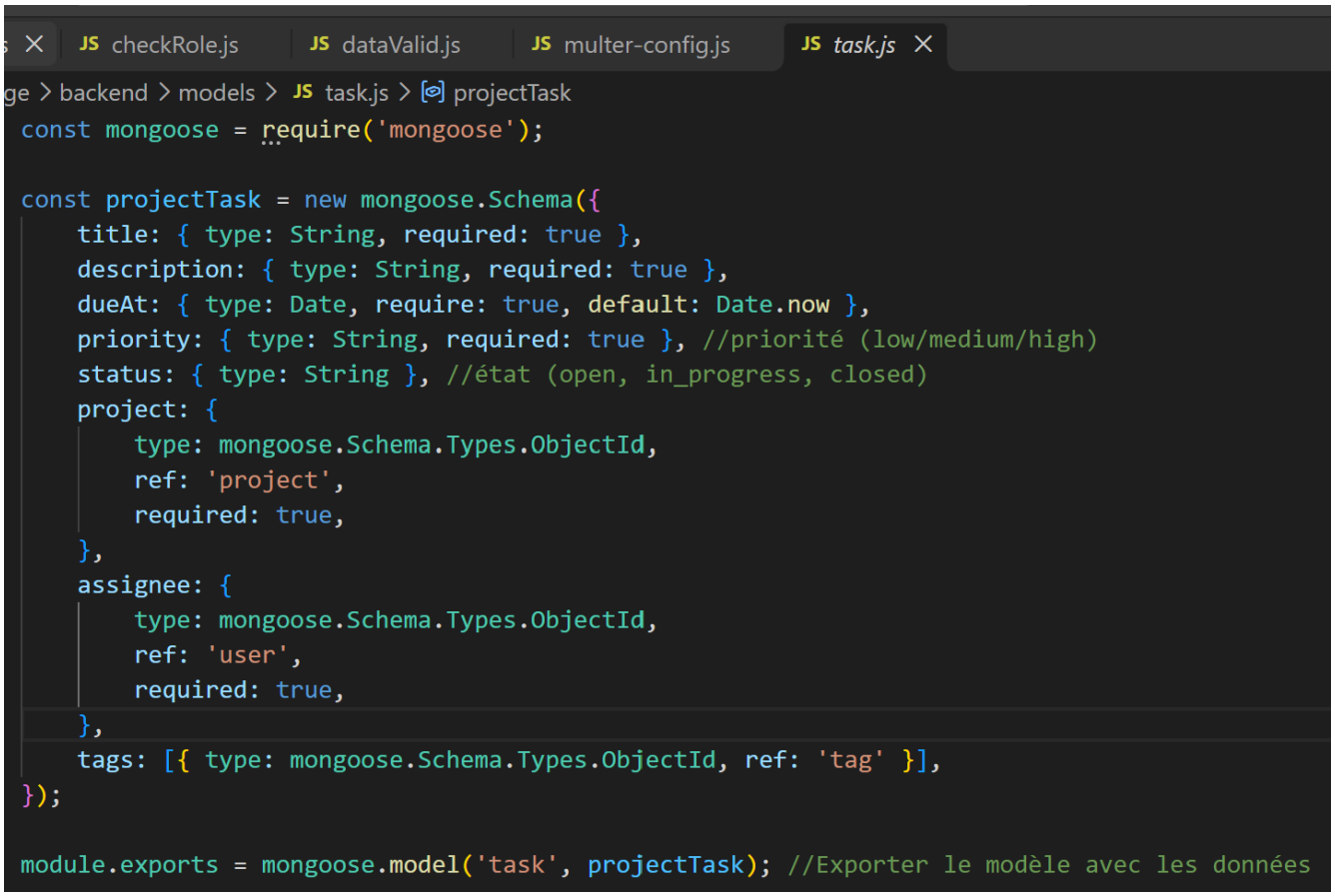
Modèle de tag :



Modèle de user :
Pour ce modèle j'ai ajouté « mongoose-unique-validator » pour éviter qu'à la création d'un compte l'application en crée deux avec la même adresse email.



Modèle de task :



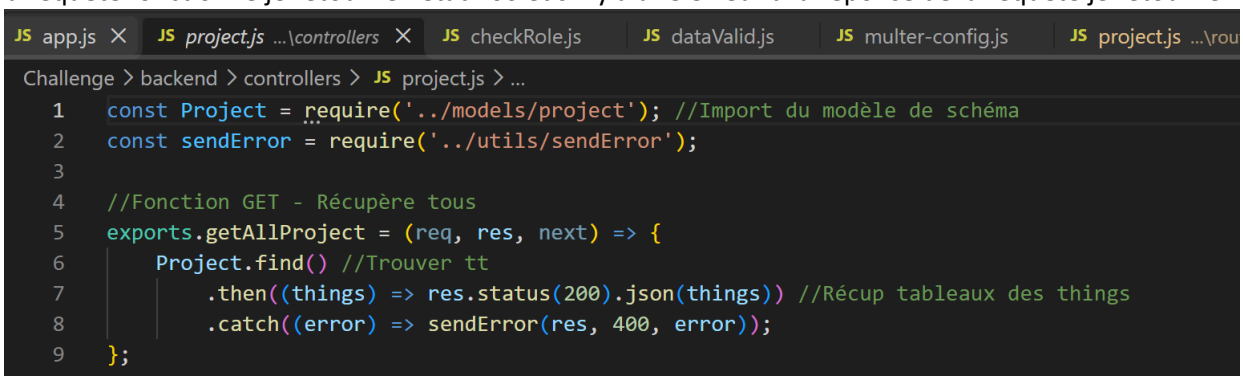
C. Controllers

Chaque controller utilise un modèle qui est importé avec la fonction require.
Exemple avec celui pour les projets.
Chaque fonction utilise la fonctionnalité exports.NOM.
Ce controller me sert à faire les actions sur le backend avec les requêtes du frontend.
J'ai créé les fonctions :

- getAllProject => récupère l'ensemble des projets
- getMyProjects => récupère tous les projets en fonction de l'ID de l'utilisateur
- createProject => crée un projet
- idProject => récupère en fonction de l'ID du projet
- modifyProject => modifie un projet
- idDeleteProject => supprime un projet
- addMembersProject => ajoute un membre au projet
- deleteMembersProject => supprime un membre du projet

Controller - getAllProject

J'utilise la propriété find pour récupérer les projets.
Si la requête fonctionne je retourne l'état 200 et s'il y a une erreur à la réponse de la requête je retourne l'état 400.



Controller - createProject

J'utilise l'instruction « new » pour créer un nouveau projet en associant le projet à son créateur.
Ensuite je le sauvegarde et le renvoie dans une réponse au format JSON avec le statut 201 si la requête fonctionne et s'il y a une erreur je retourne l'état 400.



Controller – getMyProjects

J'ai mis une pagination qui me servira à l'affichage dans le frontend que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 400.

```
// Fonction GET - Récupère les projets de l'utilisateur connecté
exports.getMyProjects = async (req, res, next) => {
    try {
        const page = parseInt(req.query.page) || 1;
        const LIMIT_SERVEUR = 4;
        const skip = (page - 1) * LIMIT_SERVEUR;

        // On définit la requête une seule fois pour l'utiliser partout
        const query = {
            $or: [{ owner: req.auth.userId }, { members: req.auth.userId }],
        };

        // On compte le nombre total de documents correspondants
        const total = await Project.countDocuments(query);

        // On récupère les projets paginés
        const projects = await Project.find(query)
            .skip(skip)
            .limit(LIMIT_SERVEUR)
            .populate('owner', 'firstName lastName') // Récupère le nom du propriétaire
            .populate('members', 'firstName lastName'); // Récupère les noms des membres

        // On renvoie le tout dans un seul objet JSON
        res.status(200).json({
            projects: projects,
            totalPages: Math.ceil(total / LIMIT_SERVEUR), // Calcul du nombre total de pages
        });
    } catch (error) {
        sendError(res, 400, error);
    }
};
```

Controller – idDeleteProject

Je cherche l'ID du projet concerné et si j'ai l'accès je peux supprimer le projet sinon je retourne l'état 401.
J'utilise l'instruction deleteOne pour supprimer un projet que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 500.
Si la requête n'arrive pas à trouver de projet ou est null, je retourne l'état 500.

```
//Fonction Delete - Supprimer en fonction de l'id
exports.idDeleteProject = (req, res, next) => {
    Project.findOne({ _id: req.params.id })
        .then((project) => {
            if (project.owner != req.auth.userId) {
                res.status(401).json({ message: 'Suppression non autorisé' });
            } else {
                Project.deleteOne({ _id: req.params.id })
                    .then(() => {
                        res.status(200).json({ message: 'Objet supprimé !' });
                    })
                    .catch((error) => sendError(res, 500, error));
            }
        })
        .catch((error) => {
            sendError(res, 500, error);
        });
};
```

Controller - idProject

J'utilise la propriété findOne pour ne retrouver qu'un seul ID que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 404.

```
//Fonction GET - Récupère en fonction de l'id
exports.idProject = (req, res, next) => {
    Project.findOne({ _id: req.params.id })
        .then((thing) => res.status(200).json(thing))
        .catch((error) => sendError(res, 404, error));
};
```

Controller – modifyProject

Je récupère le body du projet actuel, si j'ai l'accès je peux modifier le projet sinon je retourne l'état 401.
Si j'ai l'accès je met à jour avec la propriété updateOne que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 401.
Si la requête n'arrive pas à trouver de projet ou est null, je retourne l'état 400.

```
//Fonction PUT - Mettre à jour si c'est un manager
exports.modifyProject = (req, res, next) => {
    const projectObject = req.file
        ? {
            ...JSON.parse(req.body.project),
          }
        : { ...req.body };

    delete projectObject._userId;

    Project.findOne({ _id: req.params.id })
        .then((project) => {
            if (project.owner != req.auth.userId) {
                res.status(401).json({ message: 'Modification non autorisé' });
            } else {
                Project.updateOne(
                    { _id: req.params.id },
                    { ...projectObject, _id: req.params.id },
                )
                    .then(() =>
                        res.status(200).json({ message: 'Objet modifié!' }),
                    )
                    .catch((error) => sendError(res, 401, error));
            }
        })
        .catch((error) => {
            sendError(res, 400, error);
        });
};
```

Controller - addMembersProject

Je vérifie que l'utilisateur a les droits, il doit être manager pour le faire, si l'utilisateur n'a pas les droits je retourne l'état 403.

Dans la propriété updateOne j'utilise la propriété addToSet pour ajouter uniquement l'ID dans le tableau des membres que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 400.

```
//Fonction POST - Ajout d'un membre dans un projet
exports.addMembersProject = (req, res, next) => {
    Project.findOne({ _id: req.params.id })
        .then((project) => {
            // 1. Vérification des droits
            if (project.owner != req.auth.userId) {
                return res.status(403).json({ message: 'Ajout non autorisé' });
            }

            // 2. Utilisation de $addToSet pour ajouter l'ID au tableau sans doublon
            Project.updateOne(
                { _id: req.params.id },
                { $addToSet: { members: req.body.members } }, // On ajoute uniquement l'ID du nouveau memb
            )
                .then(() =>
                    res.status(200).json({ message: 'Ajout terminé !' }),
                )
                .catch((error) => sendError(res, 400, error));
        })
        .catch((error) => {
            sendError(res, 500, error);
        });
};
```

Controller - deleteMembersProject

Je vérifie que l'utilisateur a les droits, il doit être manager pour le faire, si l'utilisateur n'a pas les droits je retourne l'état 403.

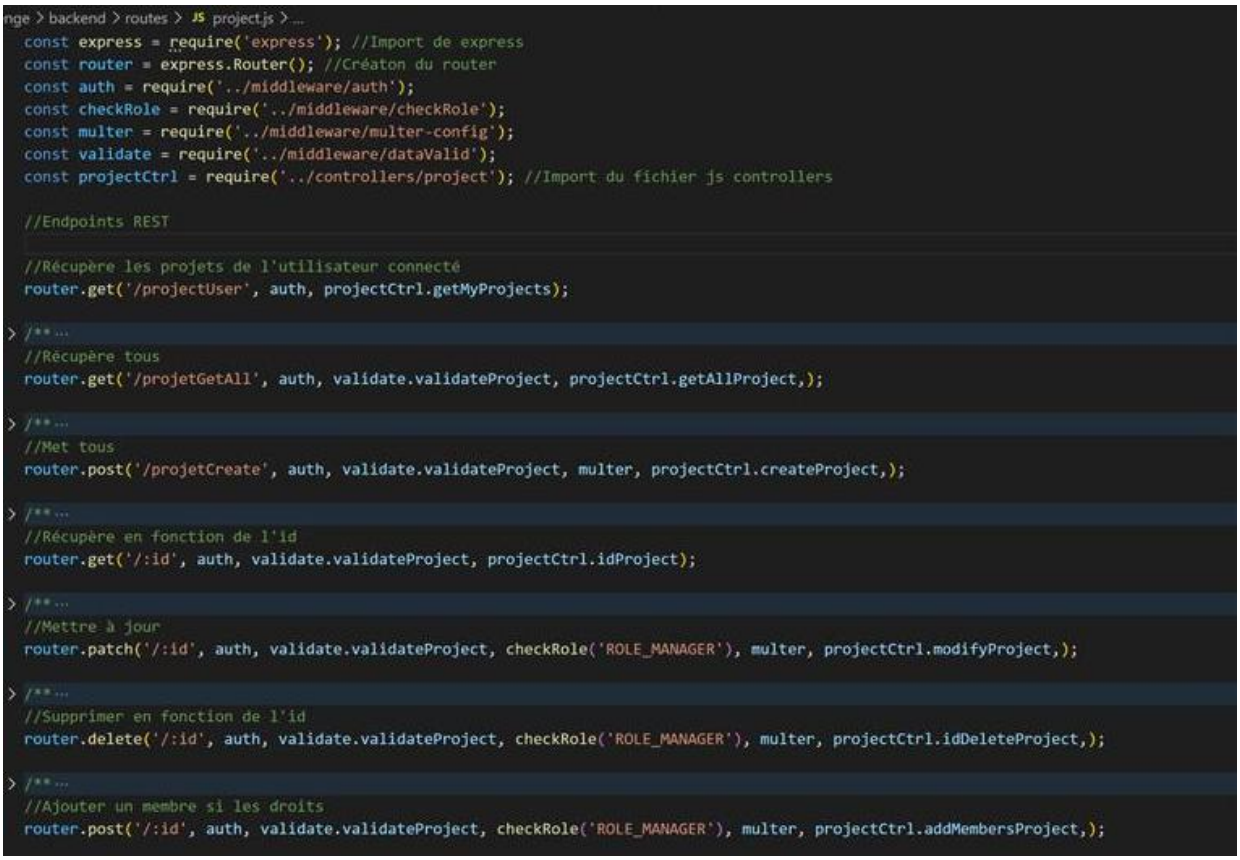
Dans la propriété updateOne j'utilise la propriété pull pour retirer uniquement l'ID du tableau des membres que je renvoie dans une réponse au format JSON avec le statut 200 si la requête fonctionne et s'il y a une erreur dans le traitement de la requête je retourne l'état 400.

```
//Fonction Delete - Suppression d'un membre dans un projet
exports.deleteMembersProject = (req, res, next) => {
    Project.findOne({ _id: req.params.id })
        .then((project) => {
            // 1. Vérification des droits
            if (project.owner != req.auth.userId) {
                return res
                    .status(403)
                    .json({ message: 'Suppression du membre non autorisé' });
            }

            // 2. Utilisation de $pull pour retirer l'ID du tableau 'members'
            Project.updateOne(
                { _id: req.params.id },
                { $pull: { members: req.body.members } }, // On ajoute uniquement l'ID du nouveau membre
            )
                .then(() =>
                    res.status(200).json({ message: 'Membre supprimé !' }),
                )
                .catch((error) => sendError(res, 400, error));
        })
        .catch((error) => {
            sendError(res, 500, error);
        });
};
```

D. Routes

Chaque route utilise un controller qui est importé avec la fonction require.
J'utilise les middlewares pour sécuriser mes routes, express pour la partie router et le CRUD dans l'utilisation de mes routes.
Exemple avec celui pour les projets.



```
nge > backend > routes > JS project.js > ...
const express = require('express'); //Import de express
const router = express.Router(); //Créaton du router
const auth = require('../middleware/auth');
const checkRole = require('../middleware/checkRole');
const multer = require('../middleware/multer-config');
const validate = require('../middleware/dataValid');
const projectCtrl = require('../controllers/project'); //Import du fichier js controllers

//Endpoints REST

//Récupère les projets de l'utilisateur connecté
router.get('/projectUser', auth, projectCtrl.getMyProjects);

> /**...
//Récupère tous
router.get('/projetGetAll', auth, validate.validateProject, projectCtrl.getAllProject,);

> /**...
//Met tous
router.post('/projetCreate', auth, validate.validateProject, multer, projectCtrl.createProject,);

> /**...
//Récupère en fonction de l'id
router.get('/:id', auth, validate.validateProject, projectCtrl.idProject);

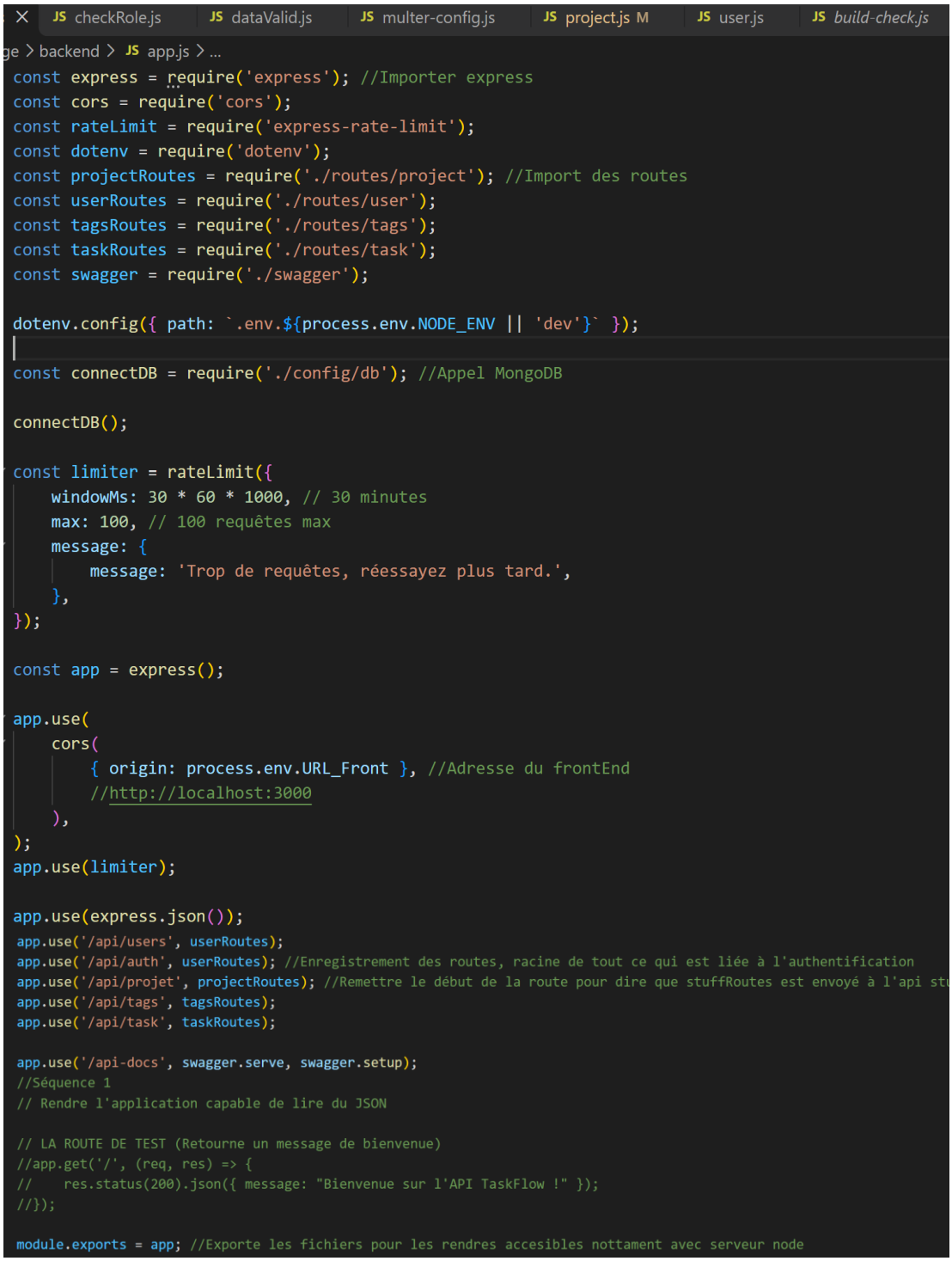
> /**...
//Mettre à jour
router.patch('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.modifyProject,);

> /**...
//Supprimer en fonction de l'id
router.delete('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.idDeleteProject,);

> /**...
//Ajouter un membre si les droits
router.post('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.addMembersProject,);
```

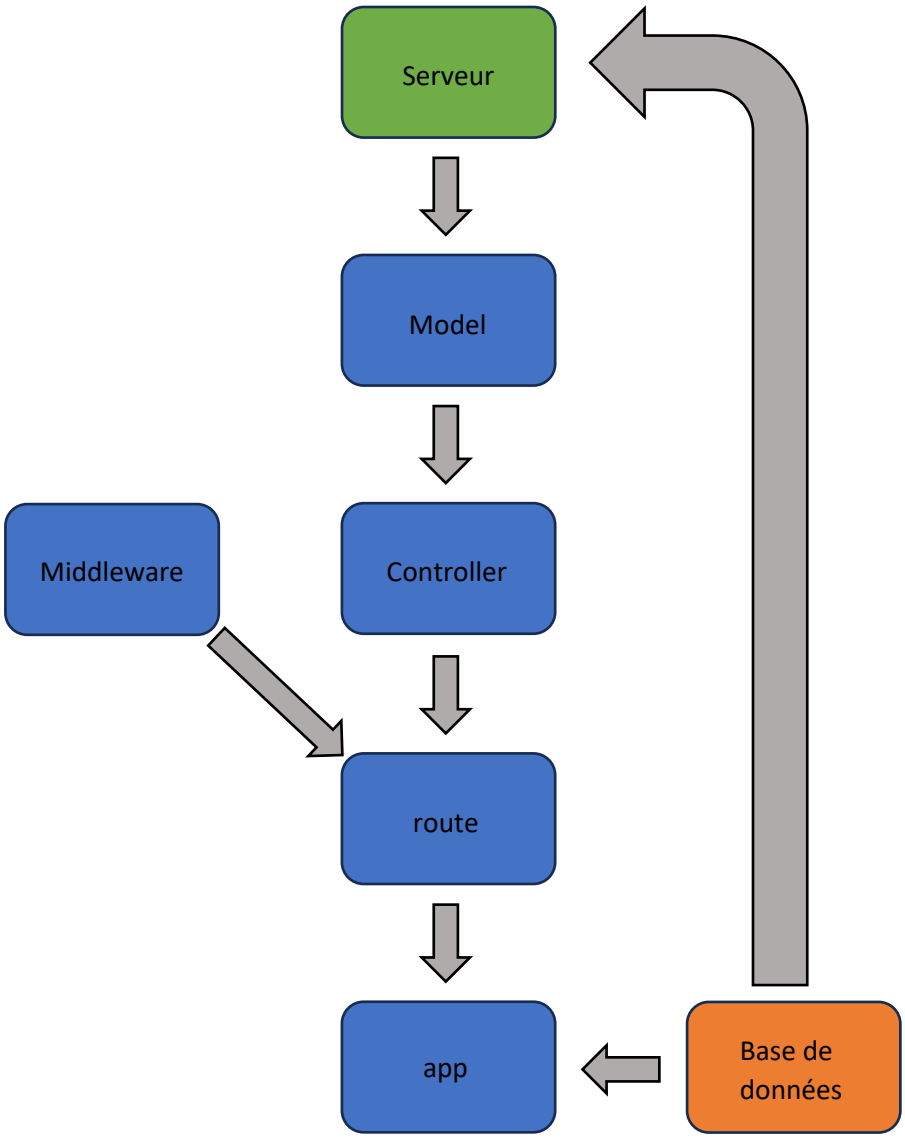
E. Application

L'application est gérée dans le fichier app.js.
La partie sécurité est abordée au chapitre III.F et la partie documentation dans le chapitre III.H.
Je connecte ma base de données puis j'utilise express dans une constante app qui me permet d'utiliser ses propriétés.
Je précise que l'application doit utiliser des fichiers json pour les rendre compatibles puis je fais l'appel des routes après avoir mis en place les sécurités avec la propriété app.use.



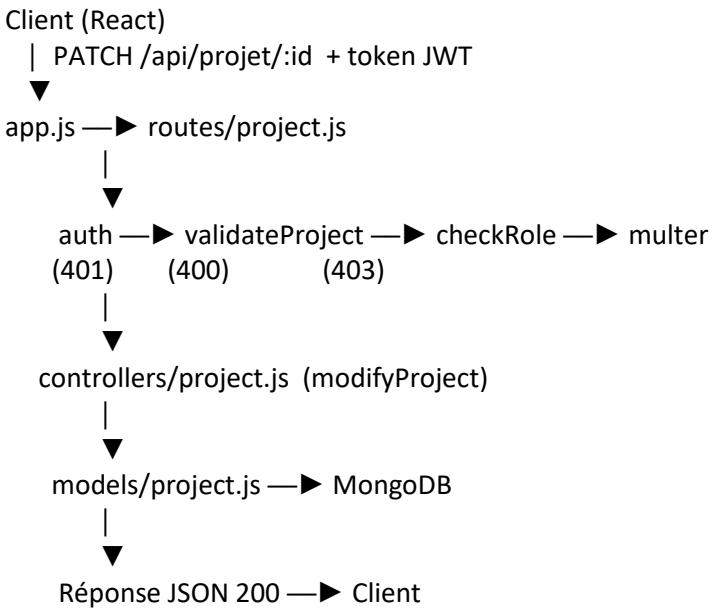
2. Couches logicielles

Voici la représentation des couches logicielles de l'application.



3. Parcours d'une requête

Exemple avec une route projet :



Cette séparation en couches permet de modifier une partie, par exemple la base de données, sans toucher aux autres. Elle rend aussi chaque couche testable séparément, et la sécurité est vérifiée avant même d'arriver à la logique métier.

F. Sécurité

L'objectif de la sécurité est de parer toute intrusion, attaque malveillante et tout vol de données en provenance de l'extérieur.
Pour cela j'ai mis en place différentes actions pour mener au mieux cette optique.

1. JWT

Dans le controller de user j'ai mis dans la fonction « login », qui me permet de me connecter, la création du token.
Le token est le badge d'identité que le serveur donne à l'utilisateur une fois qu'il s'est bien connecté. C'est lui qui fait le lien avec mon middleware auth.js.
La signature du token comporte trois paramètres :
- userId et role
- La clé secrète qui me sert à signer le token. La signature prouve que c'est bien mon serveur qui l'a émis et que personne ne l'a modifié.
Si quelqu'un change le userId dans le token, la signature ne correspond plus et jwt.verify échoue.
- L'expiration : au bout de 24h, jwt.verify rejettera le token, et l'utilisateur devra se reconnecter.

```
exports.login = (req, res, next) => {
    User.findOne({ email: req.body.email })
        .then((user) => {
            if (user === null) {
                res.status(401).json({
                    message: 'Paire identifiant/mot de passe incorrecte',
                });
            } else {
                bcrypt
                    .compare(req.body.password, user.password)
                    .then((valid) => {
                        if (!valid) {
                            res.status(401).json({
                                message:
                                    'Paire identifiant/mot de passe incorrecte',
                            });
                        } else {
                            res.status(200).json({
                                // Nécessaire pour l'auth des requêtes émises par le client
                                user: {
                                    _id: user._id,
                                    firstName: user.firstName,
                                    lastName: user.lastName,
                                    email: user.email,
                                    roles: user.roles,
                                },
                                token: jwt.sign(
                                    { userId: user._id, role: user.roles },
                                    process.env.JWT_SECRET,
                                    { expiresIn: '24h' },
                                ),
                            });
                        }
                    })
                    .catch((error) => sendError(res, 500, error));
            }
        })
        .catch((error) => sendError(res, 500, error));
};
```

2. Bcrypt

Dans le controller de user j'ai mis dans la fonction « createUser » qui me permet de créer un utilisateur, un hachage du mot de passe pour éviter que le mot de passe original soit exposé.
Dans la propriété « hash » je récupère le mot de passe dans le body, j'ai mis un salt rounds de 10 ce qui représente 2^10 = 1024 tours puis j'enregistre ce nouvel utilisateur avec son modèle et son mot de passe haché.

```
JS project.js ...\controllers   JS auth.js   JS user.js ...\controllers   JS checkRole.js   JS data
Challenge > backend > controllers > JS user.js > ...
1   const bcrypt = require('bcrypt');
2   const jwt = require('jsonwebtoken');
3   const User = require('../models/user');
4   const sendError = require('../utils/sendError');
5
6   // 1. Action pour INSÉRER un utilisateur (POST)
7   exports.createUser = (req, res, next) => {
8       // On crypte le mot de passe reçu dans le body
9       bcrypt
10          .hash(req.body.password, 10)
11          .then((hash) => {
12              const user = new User({
13                  firstName: req.body.firstName,
14                  lastName: req.body.lastName,
15                  email: req.body.email,
16                  password: hash,
17                  roles: req.body.roles || ['ROLE_USER'],
18              });
19
20              // Enregistrement dans la base MongoDB Atlas
21              user.save()
22                  .then(() =>
23                      res.status(201).json({
24                          message: 'Utilisateur créé avec succès !',
25                          data: user,
26                      }),
27                  )
28                  .catch((error) => sendError(res, 400, error));
29          })
30          .catch((error) => sendError(res, 500, error));
31  };
```

Le hachage du mot de passe a aussi été mis dans la fonction « ModifyUser » si l'utilisateur modifie son mot de passe.

```
exports.modifyUser = async (req, res, next) => {
    try {
        const userObject = req.file
            ? {
                ...JSON.parse(req.body.user),
              }
            : { ...req.body };

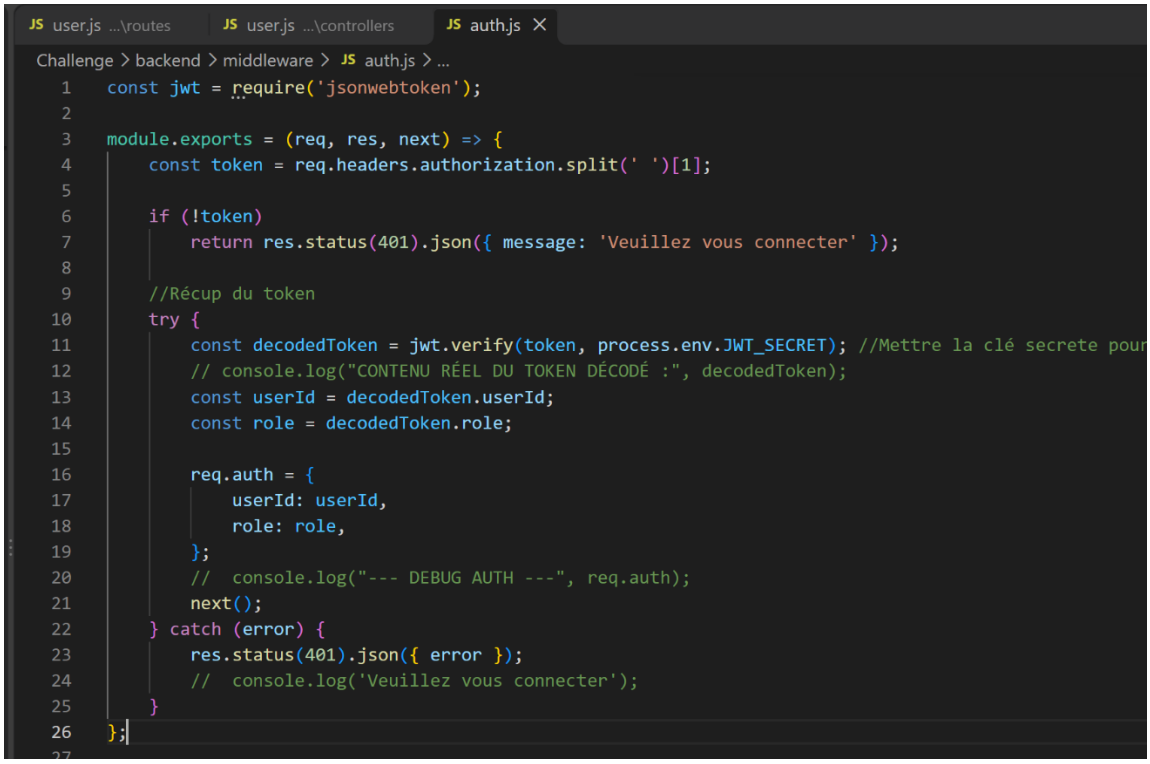
        // Si un nouveau mot de passe est fourni, on le hash avant sauvegarde
        if (userObject.password) {
            userObject.password = await bcrypt.hash(userObject.password, 10);
        } else {
            delete userObject.password;
        }

        await User.findOne({ _id: req.params.id });
        await User.updateOne(
            { _id: req.params.id },
            { ...userObject, _id: req.params.id },
        );

        res.status(200).json({ message: 'Utilisateur modifié!' });
    } catch (error) {
        sendError(res, 400, error);
    }
};
```

3. Authentification

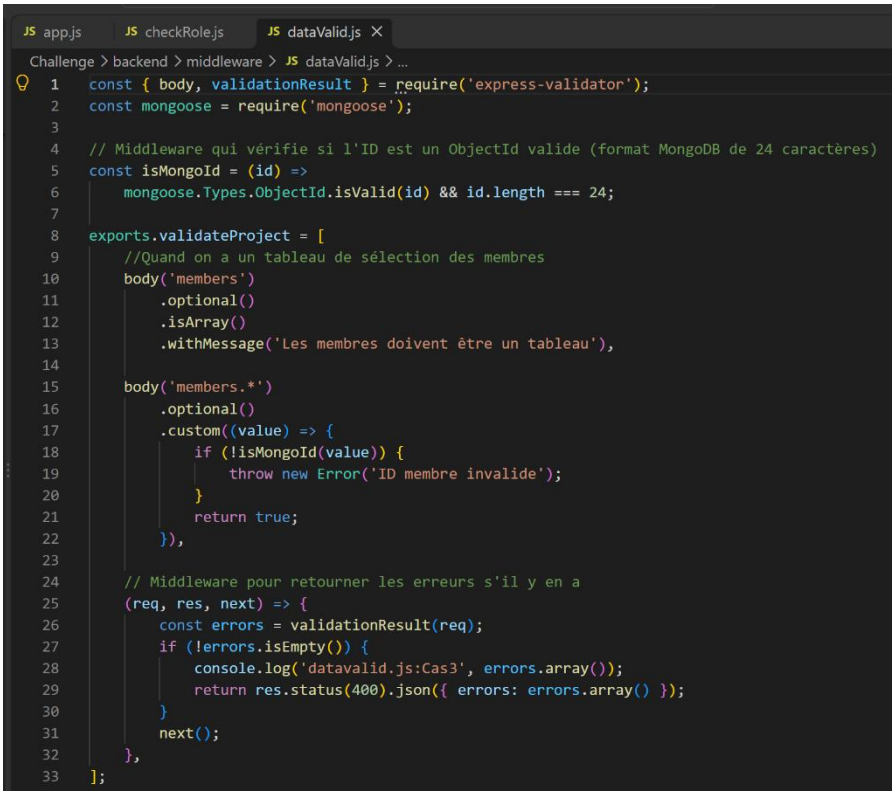
Dans le middleware auth.js je récupère le token pour vérifier les informations mises dans le token (userId et role).



```
JS user.js ...\routes   JS user.js ...\controllers   JS auth.js ×
Challenge > backend > middleware > JS auth.js > ...
1   const jwt = require('jsonwebtoken');
2
3   module.exports = (req, res, next) => {
4       const token = req.headers.authorization.split(' ')[1];
5
6       if (!token)
7           return res.status(401).json({ message: 'Veuillez vous connecter' });
8
9       //Récup du token
10      try {
11          const decodedToken = jwt.verify(token, process.env.JWT_SECRET); //Mettre la clé secrete pour
12          // console.log("CONTENU RÉEL DU TOKEN DÉCODÉ :", decodedToken);
13          const userId = decodedToken.userId;
14          const role = decodedToken.role;
15
16          req.auth = {
17              userId: userId,
18              role: role,
19          };
20          //  console.log("--- DEBUG AUTH ---", req.auth);
21          next();
22      } catch (error) {
23          res.status(401).json({ error });
24          //  console.log('Veuillez vous connecter');
25      }
26  };
```

4. Validation des données

Le format des données est contrôlé, l'ID du projet doit comporter 24 caractères, les membres doivent faire partie d'un tableau et avoir un format valide.

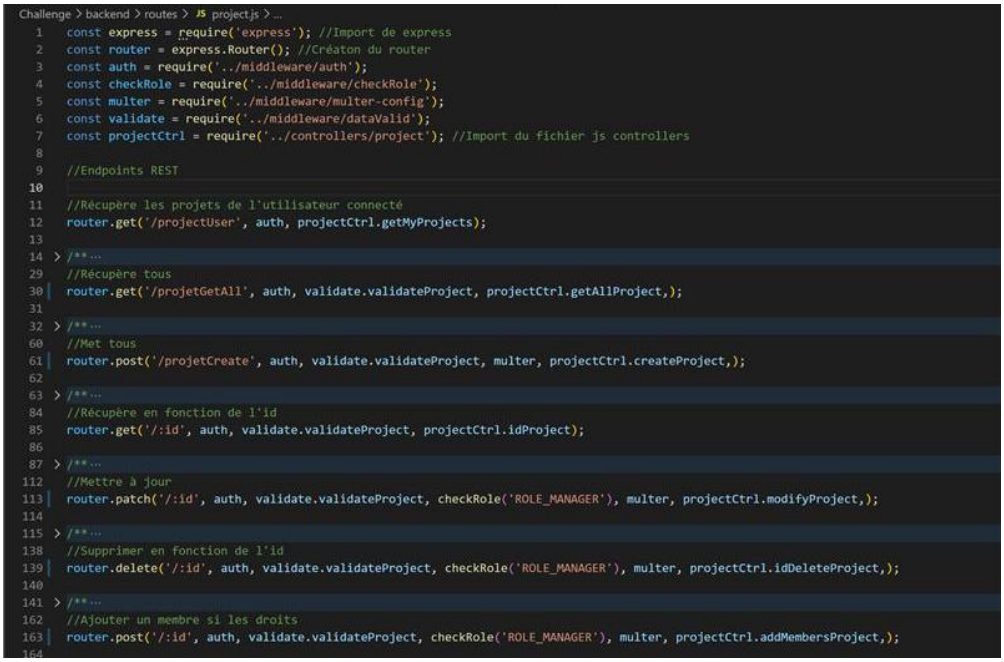


```
JS app.js   JS checkRole.js   JS dataValid.js ×
Challenge > backend > middleware > JS dataValid.js > ...
1   const { body, validationResult } = require('express-validator');
2   const mongoose = require('mongoose');
3
4   // Middleware qui vérifie si l'ID est un ObjectId valide (format MongoDB de 24 caractères)
5   const isMongoId = (id) =>
6       mongoose.Types.ObjectId.isValid(id) && id.length === 24;
7
8   exports.validateProject = [
9       //Quand on a un tableau de sélection des membres
10      body('members')
11          .optional()
12          .isArray()
13          .withMessage('Les membres doivent être un tableau'),
14
15      body('members.*')
16          .optional()
17          .custom((value) => {
18              if (!isMongoId(value)) {
19                  throw new Error('ID membre invalide');
20              }
21              return true;
22          }),
23
24      // Middleware pour retourner les erreurs s'il y en a
25      (req, res, next) => {
26          const errors = validationResult(req);
27          if (!errors.isEmpty()) {
28              console.log('datavalid.js:Cas3', errors.array());
29              return res.status(400).json({ errors: errors.array() });
30          }
31          next();
32      },
33  ];
```

5. Routes

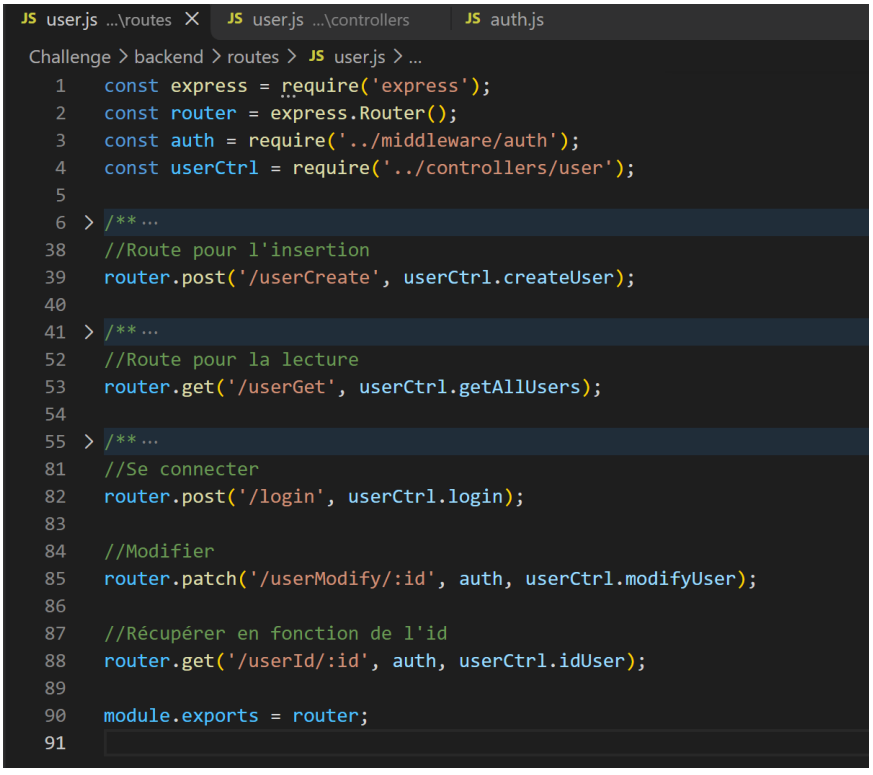
Je fais l'appel des middlewares sur les routes.
Exemple avec les middlewares utilisés pour les routes project :

- auth.js pour vérifier que le token est valide
- dataValid.js pour vérifier que l'ID comporte 24 caractères, les membres fassent partie d'un tableau et qu'ils ont tous un format valide
- checkRole avec le paramètre 'ROLE_MANAGER' pour donner accès aux modifications



```
Challenge > backend > routes > JS project.js > ...
1   const express = require('express'); //Import de express
2   const router = express.Router(); //Créaton du router
3   const auth = require('../middleware/auth');
4   const checkRole = require('../middleware/checkRole');
5   const multer = require('../middleware/multer-config');
6   const validate = require('../middleware/dataValid');
7   const projectCtrl = require('../controllers/project'); //Import du fichier js controllers
8
9   //Endpoints REST
10
11  //Récupère les projets de l'utilisateur connecté
12  router.get('/projectUser', auth, projectCtrl.getMyProjects);
13
14  /**...
29  //Récupère tous
30  router.get('/projetGetAll', auth, validate.validateProject, projectCtrl.getAllProject,);
31
32  /**...
60  //Met tous
61  router.post('/projetCreate', auth, validate.validateProject, multer, projectCtrl.createProject,);
62
63  /**...
84  //Récupère en fonction de l'id
85  router.get('/:id', auth, validate.validateProject, projectCtrl.idProject);
86
87  /**...
112 //Mettre à jour
113 router.patch('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.modifyProject,);
114
115 /**...
138 //Supprimer en fonction de l'id
139 router.delete('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.idDeleteProject,);
140
141 /**...
162 //Ajouter un membre si les droits
163 router.post('/:id', auth, validate.validateProject, checkRole('ROLE_MANAGER'), multer, projectCtrl.addMembersProject,);
164
```

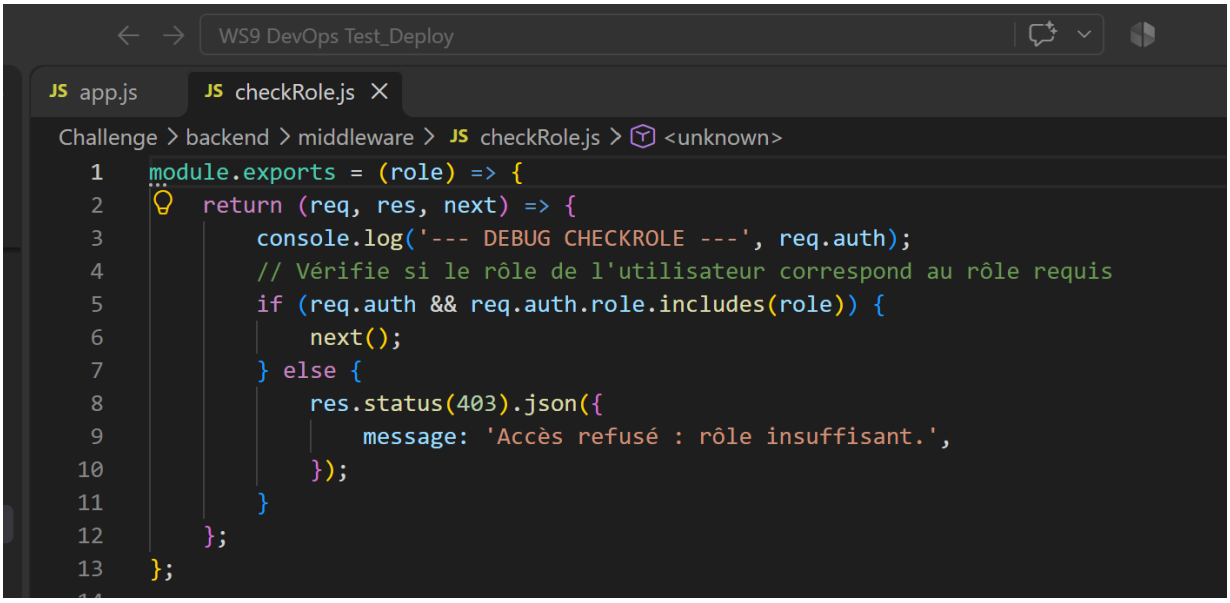
Exemple avec les routes user.
J'utilise le middleware auth.js pour vérifier que le token est valide et le controller user pour le hachage du mot de passe et le token.



```
JS user.js ...\routes  JS user.js ...\controllers  JS auth.js
Challenge > backend > routes > JS user.js > ...
1   const express = require('express');
2   const router = express.Router();
3   const auth = require('../middleware/auth');
4   const userCtrl = require('../controllers/user');
5
6   /**...
38  //Route pour l'insertion
39  router.post('/userCreate', userCtrl.createUser);
40
41  /**...
52  //Route pour la lecture
53  router.get('/userGet', userCtrl.getAllUsers);
54
55  /**...
81  //Se connecter
82  router.post('/login', userCtrl.login);
83
84  //Modifier
85  router.patch('/userModify/:id', auth, userCtrl.modifyUser);
86
87  //Récupérer en fonction de l'id
88  router.get('/userId/:id', auth, userCtrl.idUser);
89
90  module.exports = router;
91
```

6. Contrôle d'accès

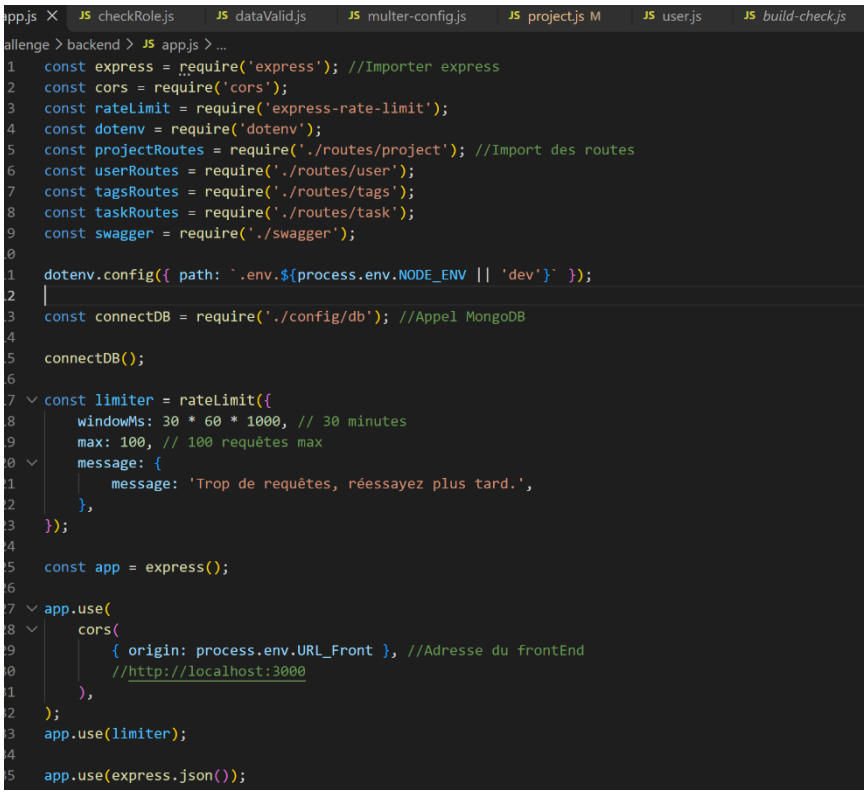
Le middleware checkRole.js vérifie que le rôle de l'utilisateur qui est manager ou non, correspond à celui paramétré dans la route.



7. Cors/rate limiter

Dans app.js j'ai ajouté deux sécurités dans un ordre précis.

1) rateLimit qui me limite le nombre de requêtes, ça m'évite une saturation du serveur. Il est paramétré à 100 requêtes maximum toutes les 30 minutes.
2) CORS me permet de n'autoriser que les requêtes venant de l'adresse de mon frontend. Cette adresse est stockée dans la variable d'environnement URL_Front.



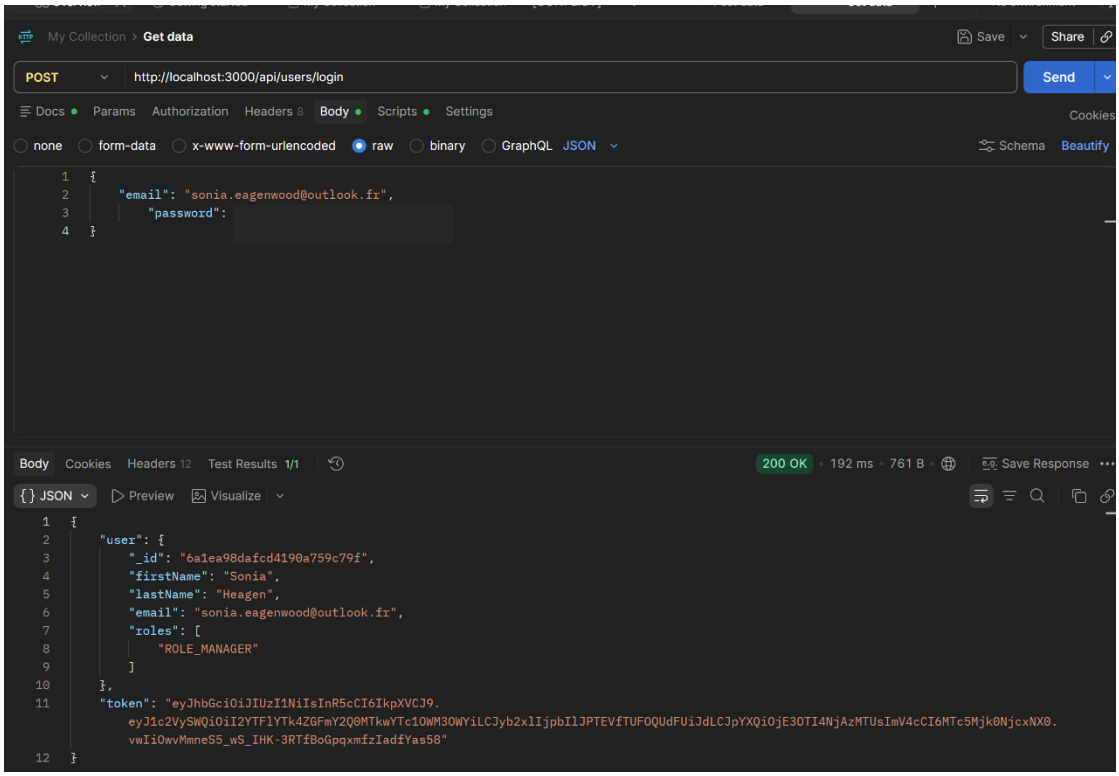
G. Tests

J'ai fait mes tests du backend en deux étapes la première pour tester manuellement l'ensemble de mes routes pour me permettre de débugger et dans un second temps j'ai ajouté les tests automatisés.

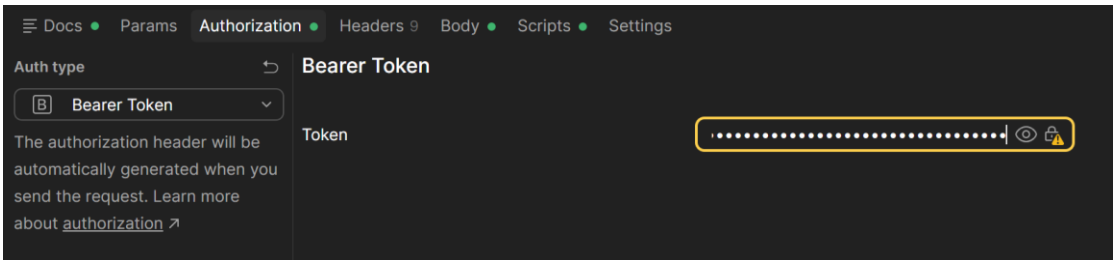
1. Postman

J'ai utilisé le logiciel Postman que j'ai aussi pu ajouter comme extension dans VSCode pour tester manuellement.
Pour utiliser l'application comme un utilisateur normal j'ai d'abord testé ma requête pour me connecter.
La requête validée m'a retourné le statut 200 et le token.

Exemple avec l'utilisateur Sonia :

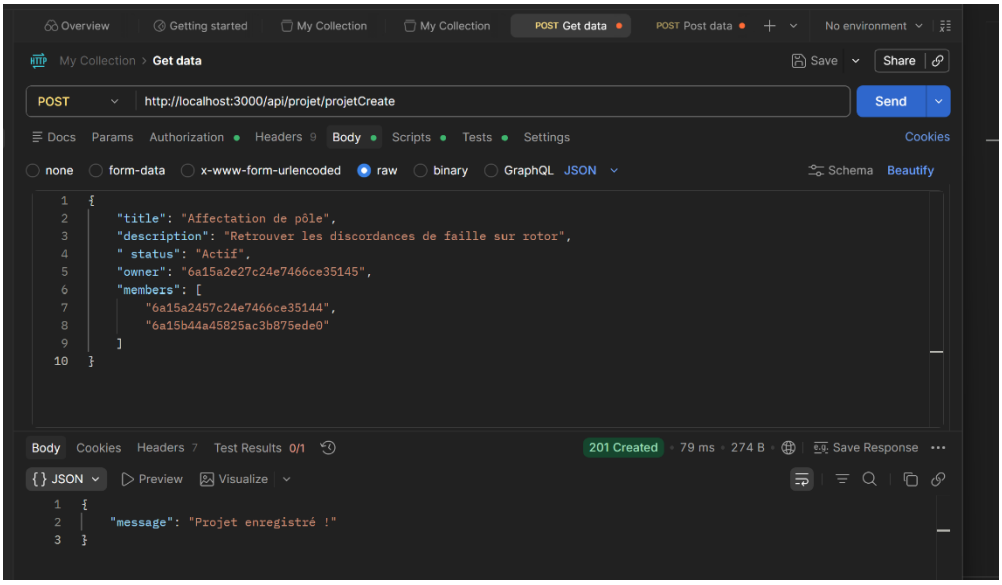


J'ai récupéré ce token puis je suis allée dans l'onglet Authorization du logiciel Postman.
Dans Auth type j'ai sélectionné Bearer Token puis j'ai mis mon token ce qui me donne les accès liés à cet utilisateur.



De cette manière j'ai pu tester chacune de mes routes

Exemple pour une requête POST sur un projet :



2. Jest/Supertest

J'ai utilisé le framework Jest et la bibliothèque Supertest pour tester en automatique.
Jest pour lancer les tests et Supertest pour envoyer de vraies requêtes HTTP à l'API.

Avant d'écrire les tests, j'ai listé les exigences les plus importantes de l'application, en commençant par la sécurité :

- L'authentification : un utilisateur avec un token invalide ou sans token ne doit pas accéder aux routes protégées ;
- Les autorisations : seul un manager peut effectuer certaines actions ;
- La validation des données : une tâche sans titre, sans priorité ou avec un identifiant de projet invalide doit être refusée ;
- Les règles métier : on ne peut pas créer deux tags identiques ;
- Les fonctionnalités principales : création de projets et de tâches, formulaires, connexion.

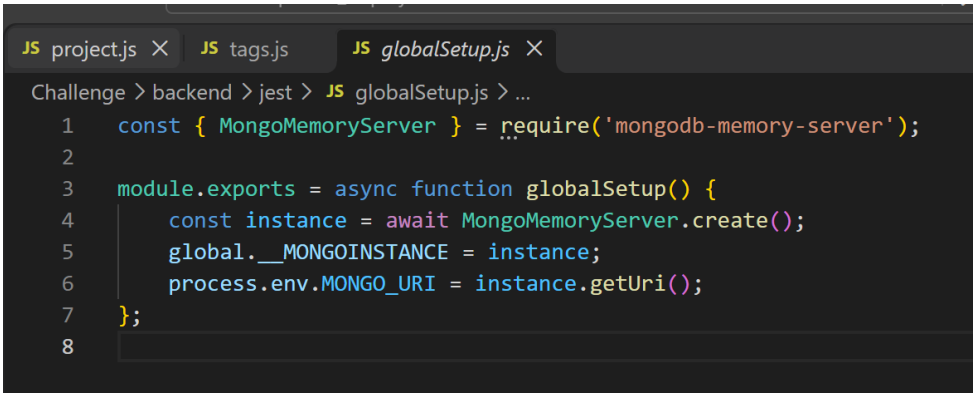
Pour chaque exigence, j'ai défini le test à écrire et le résultat attendu.

Fichier	Test	Exigence couverte
jwtInvalide.test.js	retourne 401 si le JWT est invalide	Authentification
noAcces.test.js	doit refuser un utilisateur non connecté	Accès à une route protégée
project.test.js	création de projet	Création / consultation d'un projet
roleAutor.test.js	autorise un manager	Autorisations d'accès
roleNoAutor.test.js	refuse un utilisateur sans le rôle requis	Autorisations d'accès
task.test.js	Création de tâche	Création / modification d'une tâche
datavalidation.test.js	refuse une tâche sans titre (champ obligatoire)	Validation des données
datavalidation.test.js	refuse une tâche sans priorité (champ obligatoire)	Validation des données
datavalidation.test.js	refuse une tâche avec un ObjectId de projet invalide	Validation des données
tags.test.js	ne doit pas créer deux tags identiques	Règle métier (unicité)

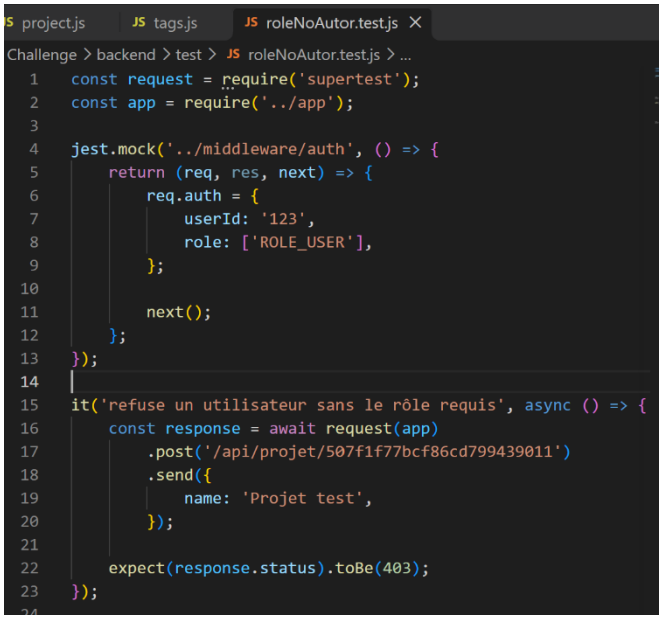
J'ai créé le fichier jest.config.js que j'ai mis à la source.
C'est le premier fichier que Jest va lire au démarrage des tests, il va me démarrer ma base de données temporaire et me l'arrêter une fois les tests terminés



Pour que les tests ne touchent jamais la vraie base de données, j'ai utilisé mongodb-memory-server.
Création de la base MongoDB temporaire avant les tests :



Exemple de test : refuser un utilisateur sans le rôle requis
Dans ce test, je simule un utilisateur connecté qui a seulement le rôle ROLE_USER.
Il essaie d'ajouter un membre à un projet, une action réservée aux managers.
Le résultat attendu est un refus avec le statut 403.



J'ai lancé les tests avec npm test dans mon terminal.

```
Test Suites: 8 passed, 8 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        25.947 s
Ran all test suites.
Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kep
t running after all tests finished?
```

H. Documentation

1. Fichier README.md

Ce fichier permet de mettre les informations relatives au projet pour permettre à n'importe qui prenant ce projet de pouvoir être autonome.

Il comporte :

- Présentation : une API REST en Node.js, Express et MongoDB pour gérer des projets, des tâches et des tags.
- Fonctionnalités : utilisateurs, authentification JWT, projets, tâches, tags, rôles, documentation Swagger et tests Jest.
- Technologies utilisées : Node.js, Express, MongoDB, Mongoose, JWT, Multer, Swagger, Jest et Supertest.
- Installation : cloner le dépôt, entrer dans le dossier, puis installer les dépendances (la liste des npm install : Express, Mongoose, bcrypt, JWT, outils de test, ESLint, Prettier, Swagger, CORS, rate-limit…).
- Configuration :
    - les deux fichiers d'environnement, .env.dev et .env.prod
    - la copie à partir de .env.example ;
    - le tableau des variables (JWT_SECRET, DB_USER, DB_PASSWORD, URL_Front) avec leur portée, privée ou publique ;
- Lancement du projet : npm run dev (avec nodemon et .env.dev) ou npm start (avec .env.prod).
- Qualité de code :
    - npm run lint (ESLint) ;
    - npm run format (Prettier) ;
    - npm run build (vérification de la syntaxe avec node --check)
    - le dossier routes/listes/, exclu car il ne contient que des notes.
- Tests Jest : la commande npm test et ce que les tests couvrent (règles métier, JWT, rôles, accès refusés, endpoints, création de projet et de tâche, validation des données).
- Documentation Swagger : l'adresse http://localhost:3000/api-docs et ce qu'on peut y faire (voir les routes, leurs paramètres et leurs réponses, et les tester depuis le navigateur).
- Routes principales : un tableau méthode + route pour les utilisateurs, les projets, les tags et les tâches.

2. Swagger

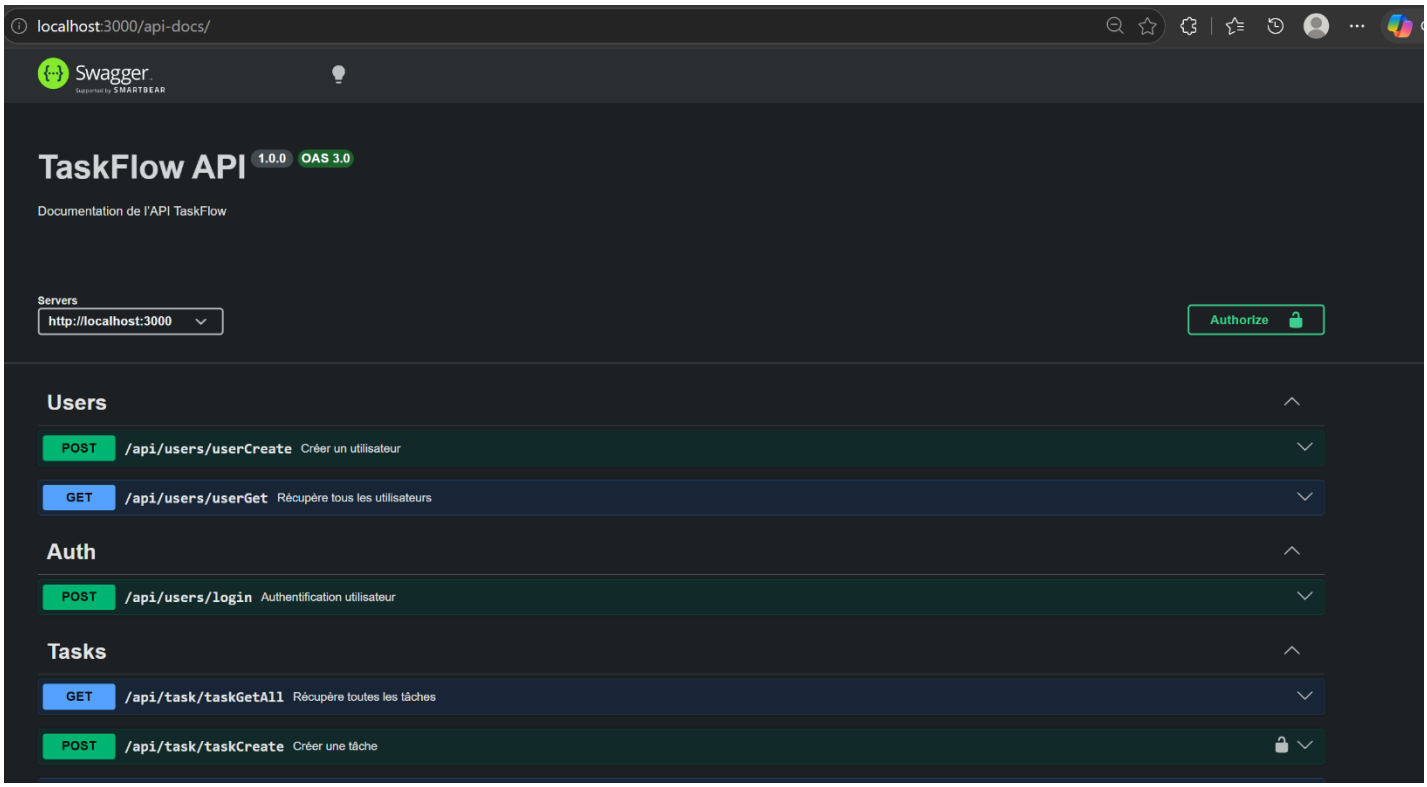
J'ai créé le fichier swagger.js, il comporte sa configuration.
Pour alimenter cette documentation, j'ai ajouté les informations au-dessus de chaque route.

```
/**
 * @swagger
 * /api/projet/projetGetAll:
 *   get:
 *     summary: Récupère tous les projets
 *     tags:
 *       - Projects
 *     security:
 *       - bearerAuth: []
 *     responses:
 *       200:
 *         description: Liste des projets récupérée avec succès
 *       401:
 *         description: Utilisateur non authentifié
 */
//Récupère tous
router.get('/projetGetAll', auth, validate.validateProject, projectCtrl.getAllProject,);
```

Je l'ai importé dans app.js et j'ai appliqué son utilisation dans l'application express à la fin du code.

```
9   const swagger = require('./swagger');
10
11  dotenv.config({ path: `.env.${process.env.NODE_ENV || 'dev'}` });
12
13  const connectDB = require('./config/db'); //Appel MongoDB
14
15  connectDB();
16
17  const limiter = rateLimit({
18      windowMs: 30 * 60 * 1000, // 30 minutes
19      max: 100, // 100 requêtes max
20      message: {
21          message: 'Trop de requêtes, réessayez plus tard.',
22      },
23  });
24
25  const app = express();
26
27  app.use(
28      cors(
29          { origin: process.env.URL_Front }, //Adresse du frontEnd
30          //http://localhost:3000
31      ),
32  );
33  app.use(limiter);
34
35  app.use(express.json());
36
37  app.use('/api/users', userRoutes);
38  app.use('/api/auth', userRoutes); //Enregistrement des routes, rac
39  app.use('/api/projet', projectRoutes); //Remettre le début de la r
40  app.use('/api/tags', tagsRoutes);
41  app.use('/api/task', taskRoutes);
42
43  app.use('/api-docs', swagger.serve, swagger.setup);
```

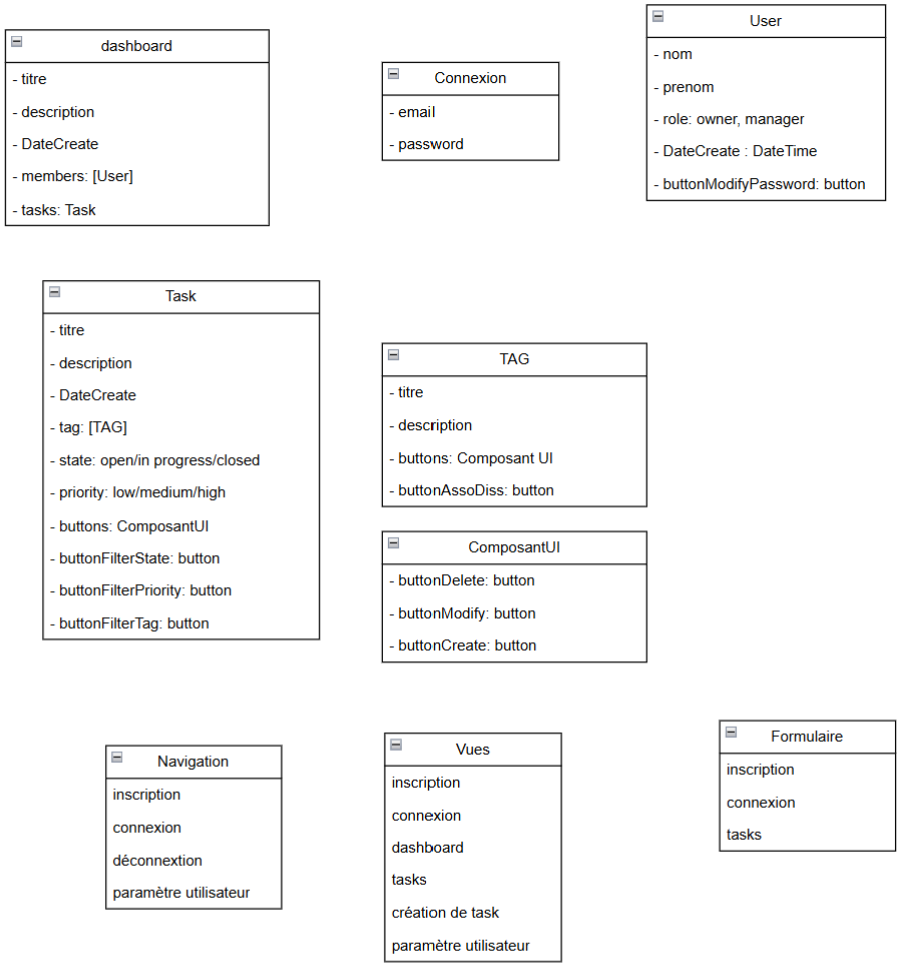
Pour accéder à la documentation il faut que le backend tourne et aller sur l'adresse http://localhost:3000/api-docs.

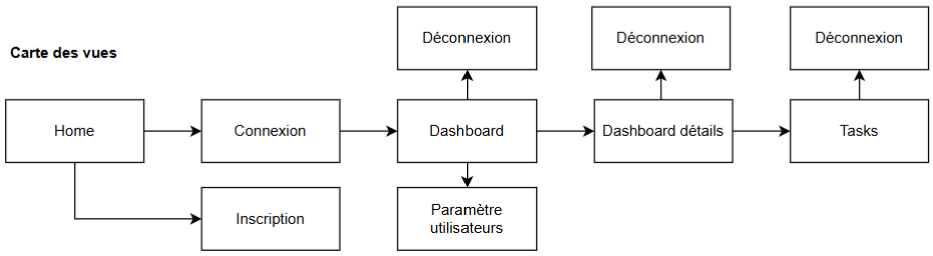


IV. Frontend

A. Analyse du besoin

J'ai fait une représentation graphique des besoins ainsi qu'une carte des vues.





B. Choix techniques

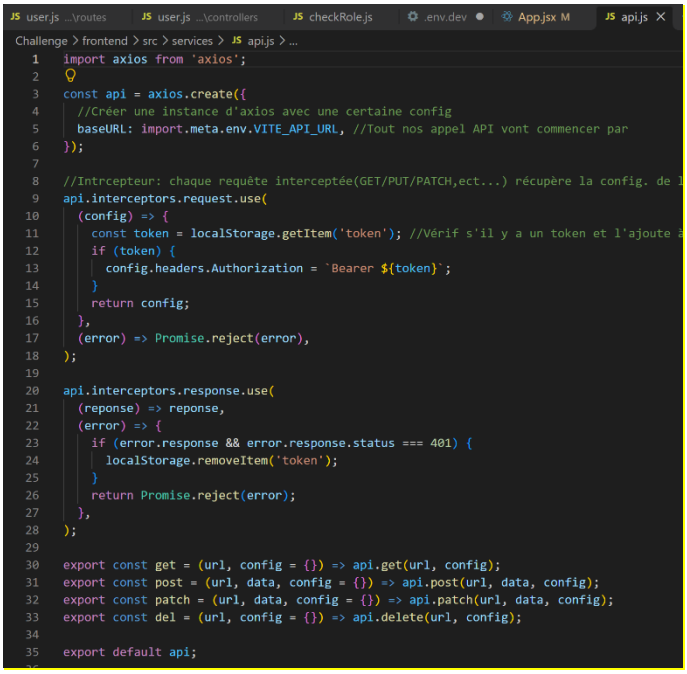
Pour le frontend, j'ai utilisé React, une bibliothèque JavaScript qui permet de construire l'interface avec des composants réutilisables. Je l'ai choisi parce qu'il me permet de découper l'interface en petits éléments (une carte projet, une carte tâche…) que je réutilise sur plusieurs pages, et de mettre à jour l'affichage automatiquement quand les données changent.
J'ai utilisé les outils suivants :

- Vite : pour créer le projet React et lancer le serveur de développement (npm run dev), qui recharge la page à chaque modification. Il produit aussi les fichiers finaux avec npm run build.
- React Router (react-router-dom) : pour naviguer entre les pages sans recharger le site.
- Axios : pour envoyer les requêtes HTTP à mon API backend.
- Tailwind CSS : pour la mise en forme, avec des classes écrites directement dans le JSX. Je l'ai chargé avec son script dans le fichier index.html.
- Vitest et React Testing Library : pour les tests automatisés.

C. Fichiers d'environnement

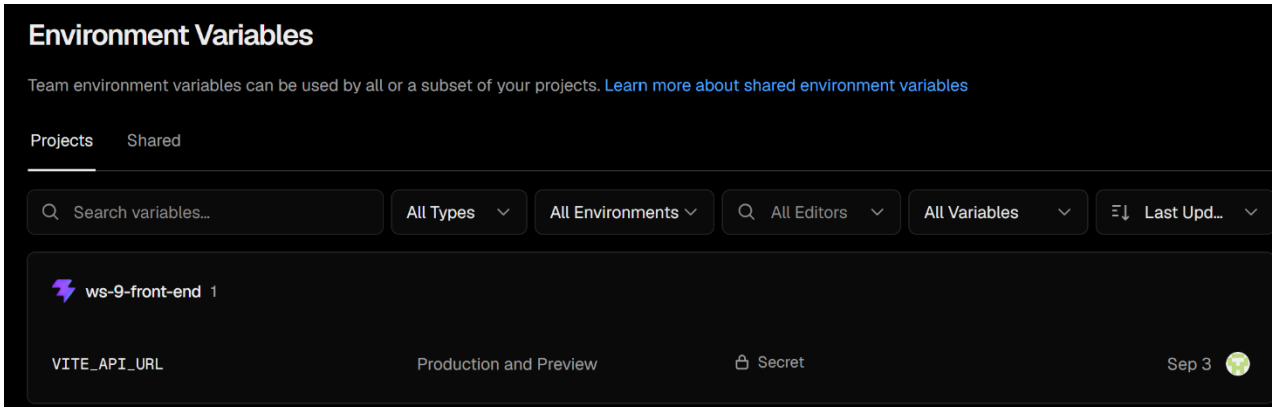
Fichier	Chargé quand
.env.development	npm run dev (Vite le charge automatiquement)
.env.production	npm run build
.env.example	Jamais, c'est le modèle

Les variables du frontend ne sont pas secrètes. Vite les copie dans le JS envoyé au navigateur, seules les variables qui commencent par VITE_ sont exposées.
Mon frontend n'a qu'une seule variable : VITE_API_URL, qui contient l'adresse de l'API backend. Vite choisit automatiquement le bon fichier : .env.development avec npm run dev et .env.production avec npm run build. Les fichiers .env.development et .env.production sont dans le .gitignore : seul le modèle .env.example est envoyé sur GitHub.
J'utilise cette variable dans le fichier services/api.js, où je crée une instance d'Axios avec l'adresse de l'API comme base (baseURL). Tous mes appels partent ainsi de cette adresse.



La différence avec le backend : au backend, les variables sont lues pendant que le serveur tourne et restent secrètes. Au frontend, Vite remplace la variable par sa valeur au moment du build, directement dans le code JavaScript. Si je change l'adresse de l'API, je dois donc refaire un build.

En production, sur Vercel, la valeur de VITE_API_URL est renseignée dans les paramètres du projet (Environment Variables) avec l'adresse de mon API hébergée sur Render.



D. Structure de l'application

1. Arborescence

J'ai organisé le dossier src en plusieurs dossiers selon le rôle de chaque fichier :

- pages : les pages complètes de l'application, une par route (Home, Login, Register, Projects, Tasks, CreateProject, EditProject…) ;
- components : les éléments réutilisables affichés dans les pages (Header, Footer, Project, Task, Tag, UserProfil) ;
- services : le fichier api.js, qui gère toutes les communications avec le backend ;
- context : la gestion de la connexion de l'utilisateur, partagée dans toute l'application (AuthContext) ;
- __tests__ : les tests automatisés.

À la racine de src, on trouve aussi main.jsx, le point d'entrée de l'application, App.jsx, qui contient les routes, et PrivateRoute.jsx, qui protège les pages réservées aux utilisateurs connectés.



2. Pages et composants

Une page correspond à un écran de l'application. Elle récupère les données auprès de l'API, puis les transmet aux composants qui les affichent.
Exemple avec la page Projects : elle récupère les projets de l'utilisateur connecté et affiche chacun d'eux avec le composant Project, sous forme de carte. J'utilise deux hooks de React :

- useState pour stocker les données affichées (la liste des projets et la page en cours de la pagination) ;
- useEffect pour lancer la requête vers l'API au chargement de la page.
-

```
function Projects() {
  const [projects, setProjects] = useState([]);
  const [pagination, setPagination] = useState({ page: 1, totalPages: 1 });

  const user = JSON.parse(localStorage.getItem('user'));

  const fetchProjects = async (page = 1) => {
    try {
      const result = await get(`api/projet/projectUser?page=${page}`);
      setProjects(result.data.projects);
      setPagination({
        page: page,
        totalPages: result.data.totalPages,
      });
    } catch (error) {
      console.error('Erreur lors de la récupération :', error);
    }
  };

  useEffect(() => {
    fetchProjects();
  }, []);
```

La page Projects utilise la pagination mise en place dans le backend : les boutons Précédent et Suivant rappellent l'API avec le numéro de page, et ils sont désactivés sur la première et la dernière page.
La page Tasks propose aussi des filtres par statut et par priorité, ainsi qu'un tri. Les choix de l'utilisateur sont envoyés à l'API dans l'adresse de la requête.

3. Navigation entre les pages

La navigation est gérée par React Router dans App.jsx. Chaque route associe une adresse à une page, par exemple /projects à la page Projects, ou /editProject/:id à la page de modification d'un projet. Le Header et le Footer sont placés en dehors des routes : ils restent affichés sur toutes les pages.

```jsx
function App() {
  return (
    <BrowserRouter>
      <div className="flex flex-col min-h-screen">
        <Header></Header>
        <main className="flex-1 max-w-4xl mx-auto p-4">
          <Routes>
            <Route path="/" element={<Home></Home>}></Route>
            <Route path="/login" element={<Login></Login>}></Route>
            <Route path="/register" element={<Register></Register>}></Route>
            <Route path="/projects" element={<PrivateRoute><Project></Project></PrivateRoute>}></Route>
            <Route path="/tasks" element={<PrivateRoute><Task></Task></PrivateRoute>}></Route>
            <Route path="/editTask/:id" element={<PrivateRoute><EditTask></EditTask></PrivateRoute>}></Route>{/*Pour modifier une tâche en cours */}
            <Route path="/editProject/:id" element={<PrivateRoute><EditProject></EditProject></PrivateRoute>}></Route>{/*Pour modifier un projet en cou
            <Route path="/editTag/:id" element={<PrivateRoute><EditTag></EditTag></PrivateRoute>}></Route>{/*Pour modifier un tag en cours*/}
            <Route path="/editUser/:id" element={<PrivateRoute><EditUser></EditUser></PrivateRoute>}></Route>{/*Pour modifier un user*/}
            <Route path="/createTask" element={<PrivateRoute><CreateTask></CreateTask></PrivateRoute>}></Route>
            <Route path="/createTag" element={<PrivateRoute><CreateTag></CreateTag></PrivateRoute>}></Route>
            <Route path="/createProject" element={<PrivateRoute><CreateProject></CreateProject></PrivateRoute>}></Route></Routes>
        </main>
        <Footer></Footer>
      </div>
    </BrowserRouter>
  );
}

export default App;
```

Pour passer des informations d'une page à l'autre, j'utilise la propriété state des liens Link. Par exemple, le bouton « Modifier » d'une carte projet envoie les informations du projet à la page EditProject, qui les récupère avec le hook useLocation (location.state). Le formulaire de modification est ainsi déjà rempli avec les valeurs actuelles.

4. Communication avec l'API

Toutes les requêtes passent par le fichier services/api.js. Il exporte quatre fonctions, get, post, patch et del, que j'utilise dans les pages au lieu d'appeler Axios directement.
J'y ai ajouté deux intercepteurs Axios :

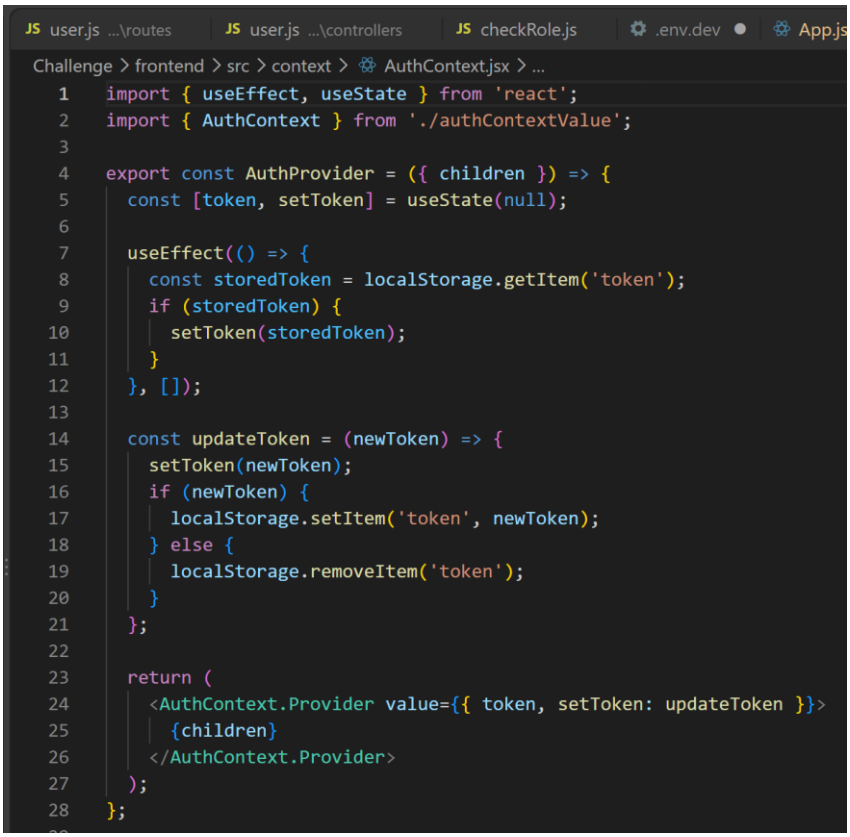
- un intercepteur de requête, qui ajoute automatiquement le token de l'utilisateur dans l'en-tête Authorization (Bearer) de chaque requête ;
- un intercepteur de réponse, qui supprime le token si l'API répond 401, c'est-à-dire si le token est invalide ou expiré.

Grâce à ça, je n'ai pas besoin d'ajouter le token à la main dans chaque page.

```
Challenge > frontend > src > services > JS apijs > ...
1  import axios from 'axios';
2
3  const api = axios.create({
4    //Créer une instance d'axios avec une certaine config
5    baseURL: import.meta.env.VITE_API_URL, //Tout nos appel API vont commencer par
6  });
7
8  //Intrcepteur: chaque requête interceptée(GET/PUT/PATCH,ect...) récupère la config. de l'appel
9  api.interceptors.request.use(
10   (config) => {
11     const token = localStorage.getItem('token'); //Vérif s'il y a un token et l'ajoute à la requêt
12     if (token) {
13       config.headers.Authorization = `Bearer ${token}`;
14     }
15     return config;
16   },
17   (error) => Promise.reject(error),
18 );
19
20 api.interceptors.response.use(
21   (reponse) => reponse,
22   (error) => {
23     if (error.response && error.response.status === 401) {
24       localStorage.removeItem('token');
25     }
26     return Promise.reject(error);
27   },
28 );
29
30 export const get = (url, config = {}) => api.get(url, config);
31 export const post = (url, data, config = {}) => api.post(url, data, config);
32 export const patch = (url, data, config = {}) => api.patch(url, data, config);
33 export const del = (url, config = {}) => api.delete(url, config);
34
35 export default api;
```

5. Gestion de la connexion

Quand l'utilisateur se connecte, la page Login envoie l'email et le mot de passe à la route POST api/users/login. Si la connexion réussit, je stocke le token et les informations de l'utilisateur dans le localStorage du navigateur, puis je le redirige vers la page de ses projets. Si les identifiants sont incorrects, l'API renvoie 401 et j'affiche le message « Email ou mot de passe incorrect. ».
Le token est partagé dans toute l'application grâce à un contexte React (AuthContext). Il est placé autour de l'application dans main.jsx, ce qui permet à n'importe quelle page de lire ou de modifier le token avec le hook useAuth.



6. Détail des pages et composants

Comme pour les controllers du backend, voici le fonctionnement des principaux éléments de l'interface. Exemple avec un projet.

A. Components - Project

Le composant Project reçoit deux informations de la page Projects, appelées props : project, qui contient les données du projet à afficher, et onUpdate, une fonction de la page.

- un clic sur la partie gauche de la carte ouvre la page Tasks, en lui transmettant l'ID et le titre du projet avec la propriété state ;
- le bouton « Terminer » envoie une requête PATCH qui passe le statut du projet à « archivé » ;
- le bouton « Modifier » ouvre la page EditProject en lui transmettant le projet ;
- le bouton « Supprimer » envoie une requête DELETE.

Après « Terminer » ou « Supprimer », le composant appelle onUpdate. La page Projects relance alors sa requête et la liste se met à jour sans recharger la page. Les données descendent de la page vers le composant par les props, et le composant prévient la page par une fonction.

```
function Project({ project, onUpdate }) {
  const deleteProject = async () => {
    await del('api/projet/' + project._id);
    onUpdate();
  };

  const markAsFinished = async () => {
    await patch('api/projet/' + project._id, { status: 'archivé' });
    onUpdate();
  };
```

B. Components – menu et déconnexion

Le Header affiche les liens de navigation. Sur mobile, un état menuOpen (useState) ouvre ou ferme le menu hamburger. Le bouton de déconnexion supprime le token du localStorage et redirige vers la page de connexion.

```
//Déconnexion
const handleLogout = () => {
  localStorage.removeItem('token');
  navigate('/login');
};

return (
  //Le code se met dedans
  <header className="bg-blue-700 text-white p-4">
    <div className="max-w-6xl mx-auto flex justify-between items-center">
      <h1 className="text-xl font-bold">TaskManager</h1>

      {/* Bouton hamburger : visible uniquement sur mobile */}
      <button className="md:hidden p-1" onClick={() => setMenuOpen((prev) => !prev)} aria-label="Ouvrir le menu">
        <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
          {menuOpen ? (
            <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M6 18L18 6M6 6l12 12"/>)
            :
            (<path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M4 6h16M4 12h16M4 18h16"/>)}
        </svg>
      </button>
```

C. Formulaire - CreateProject

Chaque champ du formulaire est relié à un état avec useState : à chaque frappe, onChange met à jour la valeur. On parle de champs contrôlés : React connaît toujours le contenu du formulaire.
À l'envoi, la fonction handleSubmit :

- bloque le rechargement de la page avec e.preventDefault() ;
- vérifie que le titre contient au moins 5 caractères, sinon elle affiche un message d'erreur et s'arrête ;
- envoie le projet à la route POST api/projet/projetCreate, avec le titre, la description, les dates, le statut, le propriétaire et les membres ;
- redirige vers la page Projects avec navigate.

Pour ajouter des membres, la page récupère au chargement la liste des utilisateurs (GET api/users/userGet). Un champ de recherche filtre cette liste par prénom ou par nom, et n'affiche pas les membres déjà sélectionnés. Un membre ajouté peut aussi être retiré avant l'envoi.

```
const handleSubmit = async (e) => {
  e.preventDefault(); //Evite le rechargement de la page

  if (title.length < 5 || description < 5) {
    setError('Veuillez mettre au minimum 5 charactères');
    return;
  } else if (status.value == '') {
    setError('Veuillez sélectionner un état');
    return;
  }

  //Appel API
  await post('api/projet/projetCreate', {
    title,
    description,
    startAt,
    endAt,
    status,
    owner: assigneeId,
    members,
  });
  //Retour vers la page tasks mais en gardant l'id du projet pour que les tâches s'affiche
  navigate('/projects');
};
```

D. Formulaire - EditProject

La page reçoit le projet à modifier par location.state. Au chargement, elle redemande les données à jour à l'API (GET api/projet/:id) et pré-remplit le formulaire. Les dates sont converties au format attendu par les champs de type date (AAAA-MM-JJ). À l'envoi, une requête PATCH enregistre les modifications, puis l'utilisateur est redirigé vers la page Projects.

```
useEffect(() => {
  get('api/projet/' + project._id).then((response) => {
    setTitle(response.data.title);
    setDescription(response.data.description);
    setStartAt(
      response.data.startAt ? response.data.startAt.split('T')[0] : '',
    );
    setEndAt(response.data.endAt ? response.data.endAt.split('T')[0] : '');
    setStatus(response.data.status);
    //  setOwner(response.data.owner);
    setMembers(response.data.members);

    if (!projectId) {
      setProjectId(response.data.project?._id || response.data.project);
    }
  });
}, [project._id]);
```

```
const handleSubmit = async (e) => {
  e.preventDefault(); //Evite le rechargement de la page
  try {
    //Appel API
    await patch('api/projet/' + project._id, {
      title,
      description,
      startAt,
      endAt,
      status,
      owner: project.owner,
      members,
    });
    navigate('/projects', {
      state: {
        projectId: projectId,
      },
    });
  } catch (error) {
    console.error('Erreur modification :', error);
  }
};
```

E. Page Tasks

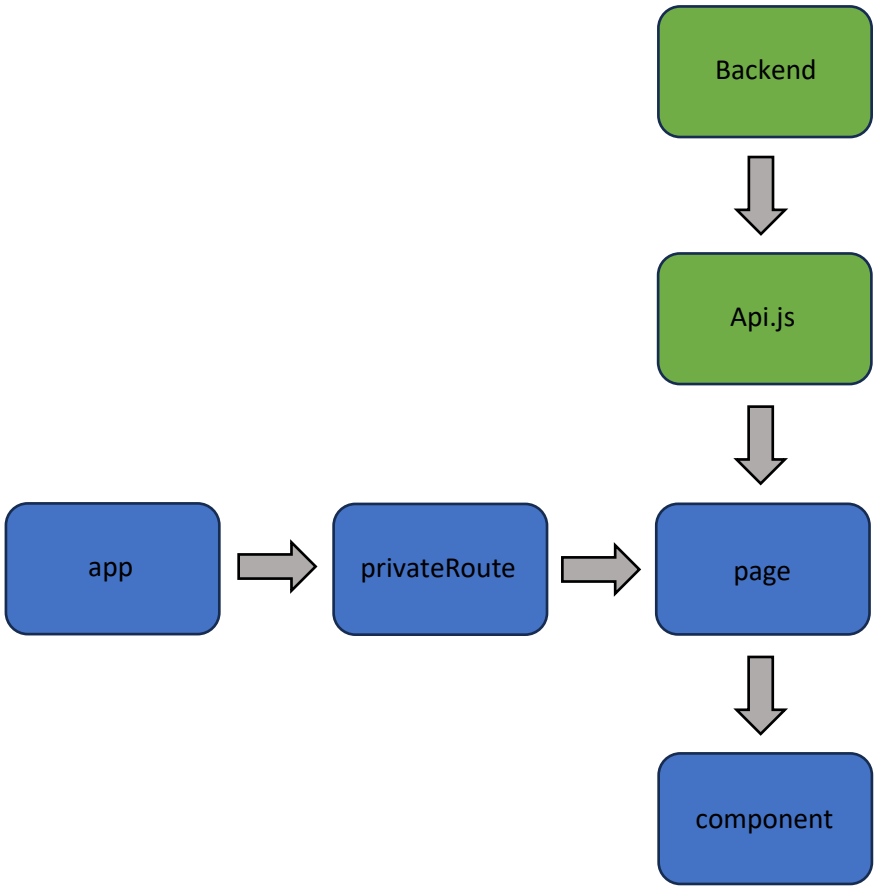
Dans la vue des tâches, j'affiche les tâches et les tags (commentaires), la page construit l'adresse de la requête avec URLSearchParams, en ajoutant seulement les filtres choisis : la page, le statut, la priorité, le critère de tri et l'ordre. Par exemple : /api/task/taskUser/ID_DU_PROJET?page=1&status=…&priority=…
Le useEffect a comme dépendances les filtres ([statusFilter, priorityFilter, sortBy, order]) : dès que l'utilisateur change un filtre, la requête est relancée automatiquement.

```
const fetchTasks = async (page = 1) => {
  try {
    // Vérification de sécurité : si on n'a pas de projectId, on arrête
    if (!projectId) {
      console.error('Aucun projectId trouvé dans le state.');
      return;
    }
    const params = new URLSearchParams({ page });
    if (statusFilter) params.set('status', statusFilter);
    if (priorityFilter) params.set('priority', priorityFilter);
    if (sortBy) {
      params.set('sortBy', sortBy);
      params.set('order', order);
    }
    // On utilise ici la variable projectId récupérée
    const result = await get(
      `/api/task/taskUser/${projectId}?${params.toString()}`,
    );
    setTasks(result.data.tasks);
    setPagination(result.data.pagination);
  } catch (error) {
    console.error('Erreur lors de la récupération :', error);
  }
};
```

7. Couches logicielles

Voici la représentation des couches logicielles de l'application.

Cette séparation a les mêmes avantages qu'au backend. Si l'adresse de l'API change, je ne modifie qu'un seul fichier (api.js).
Si je change l'apparence d'une carte, je ne touche qu'au composant, sans risquer de casser les requêtes et chaque couche peut être testée séparément : dans mes tests, je remplace la couche service (api.js) par des mocks pour tester sans le backend.



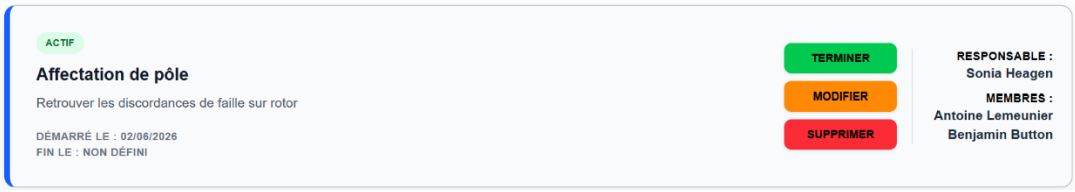
8. Parcours d'une action utilisateur

Exemple quand l'utilisateur ouvre la page de ses projets :
- l'utilisateur clique sur le lien vers /projects ;
- PrivateRoute vérifie qu'un token est présent, sinon il le redirige vers /login ;
- la page Projects s'affiche et le useEffect appelle la fonction fetchProjects ;
- services/api.js ajoute le token à la requête GET api/projet/projectUser?page=1 ;
- le backend vérifie le token, récupère les projets de l'utilisateur et renvoie une réponse au format JSON ;
- useState met à jour la liste des projets, et React affiche une carte Project pour chacun d'eux.

E. Interface utilisateur

J'ai conçu l'interface pour qu'elle soit lisible et simple à utiliser. Les projets, les tâches et les tags sont affichés sous forme de cartes, avec une bordure de couleur et une ombre pour les détacher du fond. Les boutons ont un code couleur : vert pour terminer, orange pour modifier et rouge pour supprimer.
- Responsive : sur mobile, le menu se replie dans un bouton « hamburger » et les parties des cartes s'affichent les unes sous les autres.
- Mode sombre : l'interface s'adapte au thème choisi dans les paramètres du navigateur (prefers-color-scheme).
- Accessibilité : j'ai évalué mes pages avec l'extension WAVE, qui signale les problèmes d'accessibilité (contrastes, champs sans étiquette…), pour pouvoir améliorer mon rendu.



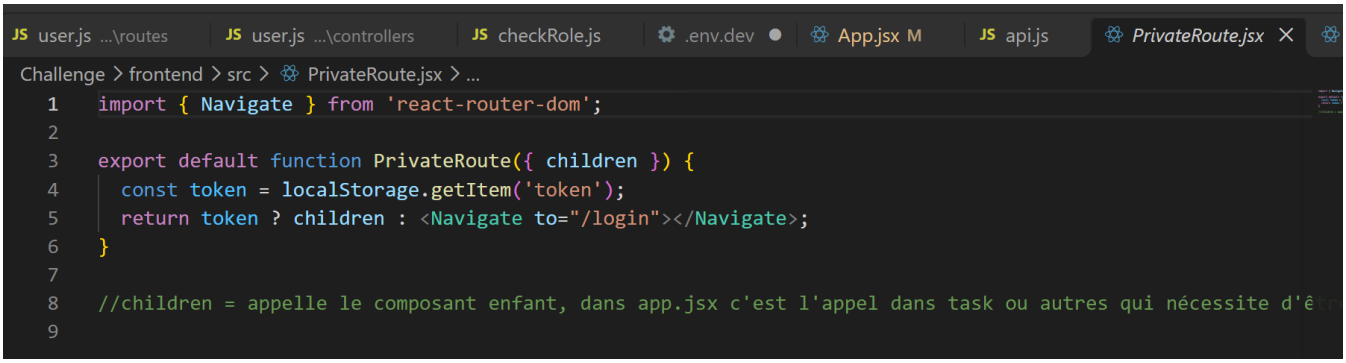




F. Sécurité

Côté frontend, j'ai mis en place plusieurs protections :

- Routes protégées : le composant PrivateRoute vérifie qu'un token est présent avant d'afficher une page réservée. Sinon, l'utilisateur est redirigé vers la page de connexion.
- Token envoyé automatiquement : l'intercepteur Axios ajoute le token à chaque requête, et le supprime si l'API répond 401.
- Aucun secret dans le frontend : la seule variable d'environnement est l'adresse de l'API, car tout ce qui se trouve dans le frontend peut être lu dans le navigateur.
- Protection contre l'injection de code (XSS) : React échappe automatiquement le texte affiché dans le JSX. Si un utilisateur saisit du code HTML ou JavaScript dans le titre d'un projet, il s'affiche comme du texte et n'est pas exécuté.



La vraie sécurité reste côté backend : PrivateRoute ne fait que cacher les pages. Même si quelqu'un contournait le frontend, l'API refuserait toute requête sans token valide (401) ou sans les droits nécessaires (403).

G. Tests

J'ai testé le frontend avec Vitest et React Testing Library. React Testing Library affiche un composant dans un faux navigateur (jsdom), puis simule les actions de l'utilisateur : remplir un champ, cliquer sur un bouton.
Pour ne pas dépendre du backend pendant les tests, je remplace les appels à l'API par de fausses fonctions (des mocks) avec vi.mock. Je peux ainsi choisir la réponse de l'API, par exemple une connexion réussie ou une erreur 401.
J'ai écrit 10 tests répartis dans 8 fichiers :

- Login : le formulaire envoie les bonnes valeurs, et un message d'erreur s'affiche si les identifiants sont incorrects.
- PrivateRoute : redirection vers /login sans token, affichage de la page avec un token ;
- Création et modification d'un projet, d'une tâche et d'un tag : les formulaires envoient les bonnes valeurs à l'API.

Exemple avec le test d'erreur de connexion : je simule une erreur 401 de l'API, je remplis le formulaire avec de mauvais identifiants, je clique sur « Se connecter » et je vérifie que le message « Email ou mot de passe incorrect. » s'affiche.

```
  it('Affiche une erreur si les identifiants sont incorrects', async () => {
    mockPost.mockRejectedValueOnce({ response: { status: 401 } });

    render(<Login></Login>, { wrapper: MemoryRouter });

    fireEvent.change(screen.getByPlaceholderText('Email'), {
      target: { value: 'mauvais@123.fr' },
    });

    fireEvent.change(screen.getByPlaceholderText('Mot de passe'), {
      target: { value: 'faux-mdp' },
    });

    fireEvent.click(screen.getByText('Se connecter'));

    await waitFor(() => {
      expect(
        screen.getByText('Email ou mot de passe incorrect.'),
      ).toBeInTheDocument();
    });
  });
});
```

Je lance les tests avec npm run test : les 10 tests passent.

```
✓ src/__tests__/CreateTag.test.jsx (1 test) 95ms
✓ src/__tests__/login.test.jsx (2 tests) 111ms
✓ src/__tests__/EditTag.test.jsx (1 test) 100ms

Test Files  8 passed (8)
     Tests  10 passed (10)
  Start at  12:57:42
  Duration  31.84s (transform 960ms, setup 20.77s, import 1.45s, tests 882ms, environment 99.16s)

PASS  Waiting for file changes...
      press h to show help, press q to quit
```

H. Documentation

J'ai rédigé un fichier README.md pour le frontend. Il présente l'architecture et les choix techniques, l'authentification, les routes, la variable d'environnement VITE_API_URL, l'installation et le lancement du projet, les tests, les commandes de qualité de code (ESLint et Prettier) et la construction de la version de production.

V. Docker

Pour que l'application fonctionne de la même façon sur n'importe quelle machine, je l'ai conteneurisée avec Docker. Un conteneur embarque l'application avec tout ce dont elle a besoin (la bonne version de Node.js, les dépendances…).

A. Dockerfile du backend

Le Dockerfile est la recette qui permet de construire l'image Docker de l'application.
Pour le backend, j'ai écrit un Dockerfile multi-étapes : chaque étape (stage) prépare une partie, et l'image finale ne garde que le nécessaire.

- deps : installe uniquement les dépendances de production (npm install --omit=dev) ;
- build : copie le code de l'application ;
- development : ajoute les outils de développement (nodemon) et lance npm run dev ;
- production : repart d'une image Node.js légère (alpine), récupère les dépendances et le code des étapes précédentes, puis lance npm run start.

Pour la sécurité, l'application tourne avec un utilisateur sans droits d'administration (nodejs) et non avec l'utilisateur root. Si quelqu'un exploitait une faille dans l'application, il n'aurait pas les droits administrateur dans le conteneur.

```
# ========================================
# Production Stage
# ========================================
ARG NODE_VERSION=24.11.1-alpine
FROM node:${NODE_VERSION} AS production

# Set working directory
WORKDIR /app

# Create non-root user for security
RUN addgroup -g 1001 -S nodejs && \
    adduser -S nodejs -u 1001 -G nodejs && \
    chown -R nodejs:nodejs /app

# Set optimized environment variables
ENV NODE_ENV=production \
    NODE_OPTIONS="--max-old-space-size=256 --no-warnings" \
    NPM_CONFIG_LOGLEVEL=silent

# Copy production dependencies from deps stage
COPY --from=deps --chown=nodejs:nodejs /app/node_modules ./node_modules
COPY --from=deps --chown=nodejs:nodejs /app/package*.json ./
# Copy built application from build stage
COPY --from=build --chown=nodejs:nodejs /app/ ./

# Switch to non-root user for security
USER nodejs

# Expose port
EXPOSE 3000

# Start production server
CMD ["npm", "run", "start"]
```

B. Dockerfile du frontend

Le Dockerfile du frontend a deux étapes :

- la première utilise Node.js pour construire l'application React avec npm run build. C'est à ce moment que la variable VITE_API_URL est intégrée dans le code ;
- la seconde utilise un serveur nginx, qui sert uniquement les fichiers produits (HTML, CSS, JavaScript).

Une fois construite, l'application React n'a plus besoin de Node.js : nginx est plus léger et plus rapide pour servir ces fichiers. L'image finale est donc beaucoup plus petite.

Challenge > frontend > Dockerfile > ...

```
# ---- Étape 1 : build (Node, produit les fichiers statiques) ----
FROM node:20-alpine AS build
WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

# URL de l'API, gravée dans le JS au moment du build (doit être joignable
# depuis le navigateur, pas depuis le réseau interne Docker)
ARG VITE_API_URL=http://localhost:3000/
ENV VITE_API_URL=$VITE_API_URL

RUN npm run build

# ---- Étape 2 : diffusion (nginx sert les fichiers statiques) ----
FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

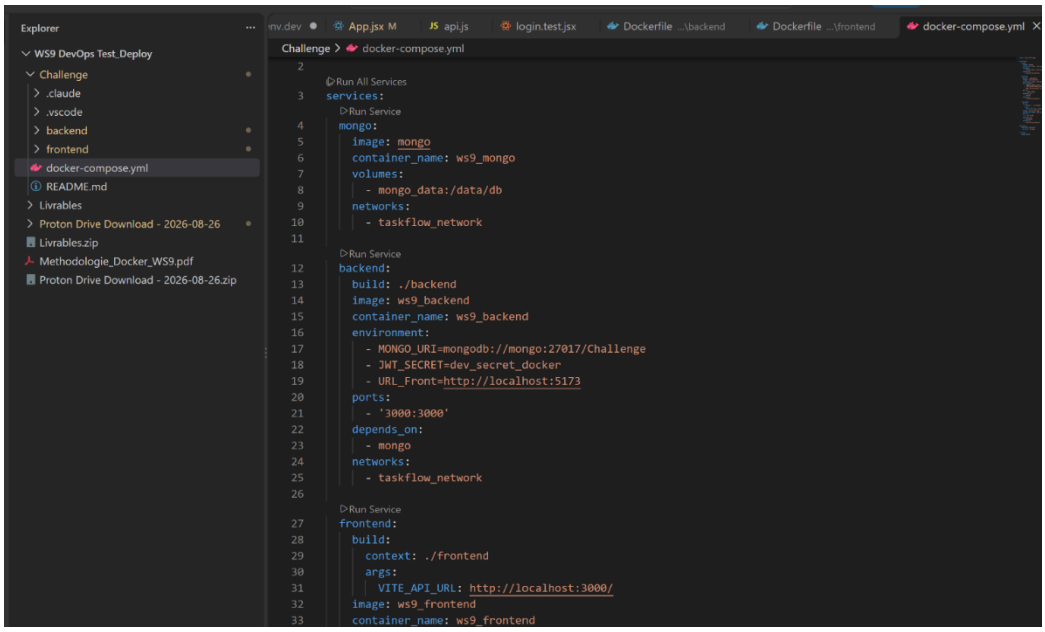
CMD ["nginx", "-g", "daemon off;"]
```

C. Docker Compose

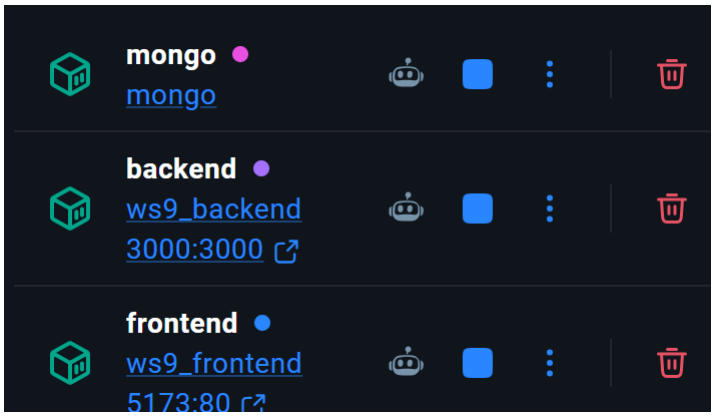
Le fichier docker-compose.yml, placé à la racine du projet, lance les trois services de l'application en une seule commande (docker compose up --build) :

- mongo : une base MongoDB locale.
- backend : l'API, accessible sur le port 3000.
- frontend : l'application React servie par nginx, accessible sur le port 5173.

Les trois conteneurs communiquent par un réseau interne (taskflow_network). La base de données n'est pas exposée à l'extérieur : seuls les autres conteneurs peuvent y accéder.
Le mot-clé depends_on fixe l'ordre de démarrage : la base, puis le backend, puis le frontend.



Docker Desktop avec les 3 conteneurs lancés.

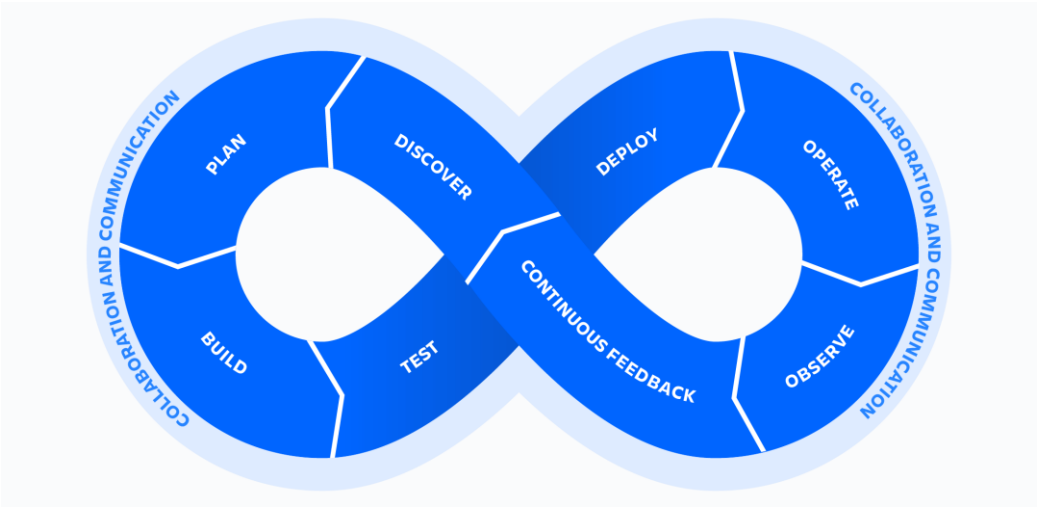


D. Fichier .dockerignore

Comme le .gitignore pour Git, le fichier .dockerignore liste les fichiers à ne pas copier dans l'image : node_modules (les dépendances sont réinstallées dans le conteneur), le dossier .git et surtout les fichiers .env. Les secrets ne se retrouvent donc jamais dans l'image Docker.

VI. DevOps

La démarche DevOps est une méthode de travail collaborative, organisée en un cycle de vie de huit phases qui permet de mieux communiquer et collaborer.



Liste des phases :

- Discover : exposition des idées, composition de l'équipe, explorer
- Plan : organiser le travail pour permettre d'améliorer la vitesse et la qualité
- Build : versionning, gestion des dépôts
- Test : intégration continue (CI) avec automatisation des tests une fois mergé
- Deploy : déploiement continu (CD) avec automatisation des déploiements une fois la partie CI validée
- Operate : gestion et maintenance des services
- Observe : identifier et résoudre les problèmes qui ont réussi à passer malgré la démarche
- Continuous feedback : évaluer chaque version et faire les rapports pour assurer le suivi auprès du client

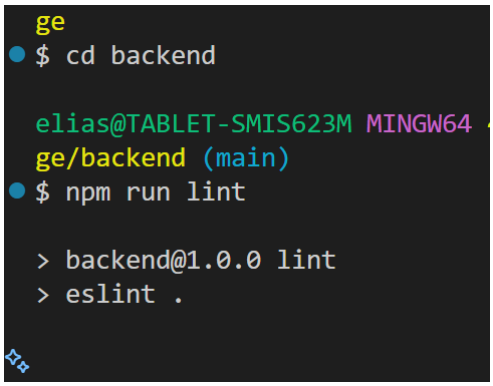
Dans ce projet je n'ai pas pu travailler en collaboration mais j'ai pu appliquer l'ensemble de la démarche.

A. Qualité du code

Avant d'automatiser le déploiement, j'ai mis en place des outils de qualité dans le backend et dans le frontend :

- ESLint (npm run lint) : analyse le code et signale les erreurs et les mauvaises pratiques, par exemple une variable déclarée mais jamais utilisée. Sa configuration se trouve dans le fichier eslint.config.js ;
- Prettier (npm run format) : met en forme le code de manière uniforme (guillemets simples, points-virgules, indentation), selon les règles du fichier .prettierrc ;
- build (npm run build) : pour le frontend, Vite construit la version de production. Pour le backend, qui n'a pas d'étape de compilation, j'ai écrit le script scripts/build-check.js : il vérifie que chaque fichier JavaScript est syntaxiquement valide avec node --check.

J'ai corrigé les problèmes relevés par ESLint et supprimé le code inutilisé.

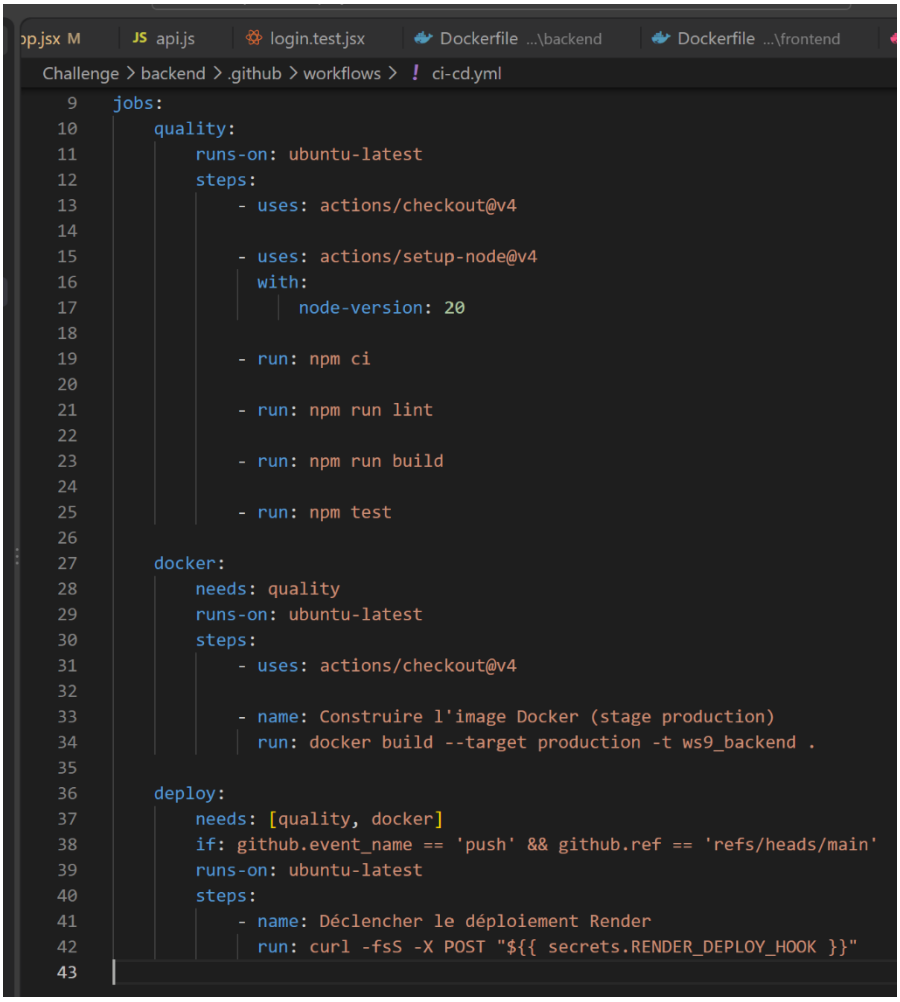


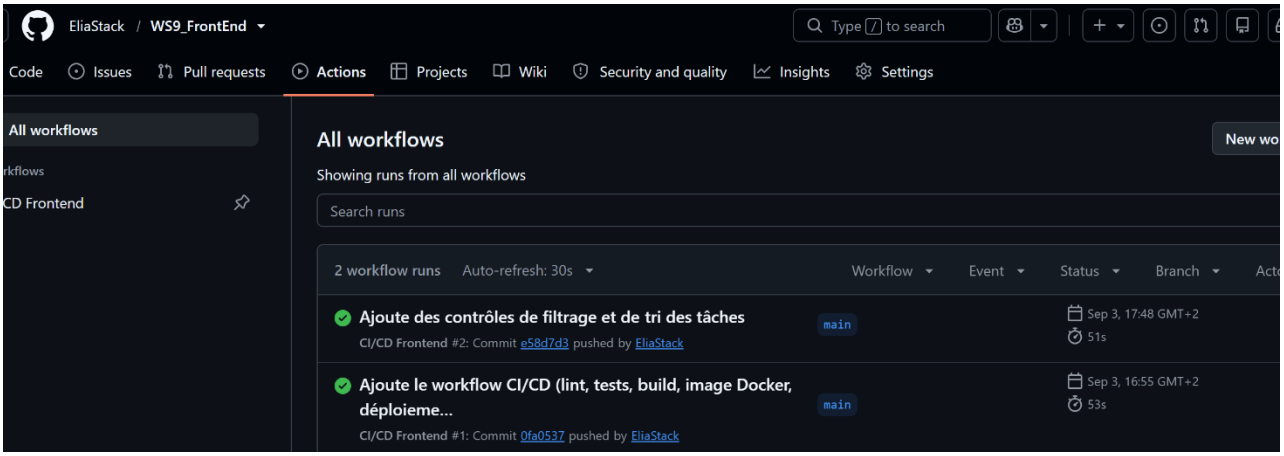
B. Pipeline CI/CD avec GitHub Actions

J'ai créé un workflow GitHub Actions (.github/workflows/ci-cd.yml) dans chacun de mes deux dépôts, WS9_BackEnd et WS9_FrontEnd. Il se déclenche à chaque push ou pull request sur la branche main. Le pipeline est découpé en trois jobs qui s'enchaînent :

- quality : installation des dépendances (npm ci), vérification du code avec ESLint, exécution des tests et build ;
- docker : construction de l'image Docker, pour vérifier qu'elle fonctionne avant la mise en ligne ;
- deploy : déclenchement du déploiement sur Render pour le backend, et sur Vercel pour le frontend.

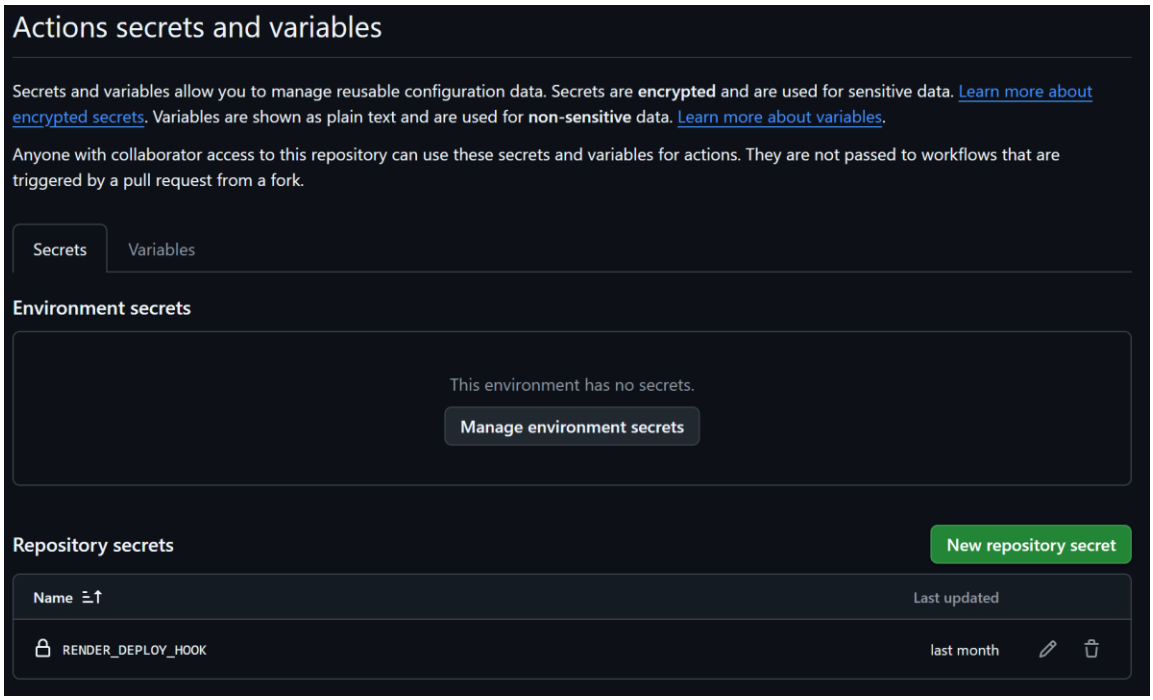
J'ai relié les jobs avec le mot-clé needs : le job docker ne démarre que si quality a réussi, et deploy seulement si les deux précédents ont réussi. Une seule erreur arrête donc toute la chaîne, et une version défectueuse ne peut pas arriver en production. Le job deploy a aussi une condition : il ne s'exécute que lors d'un push sur main. Une pull request lance les vérifications, mais ne déploie rien.





C. Sécurisation du déploiement

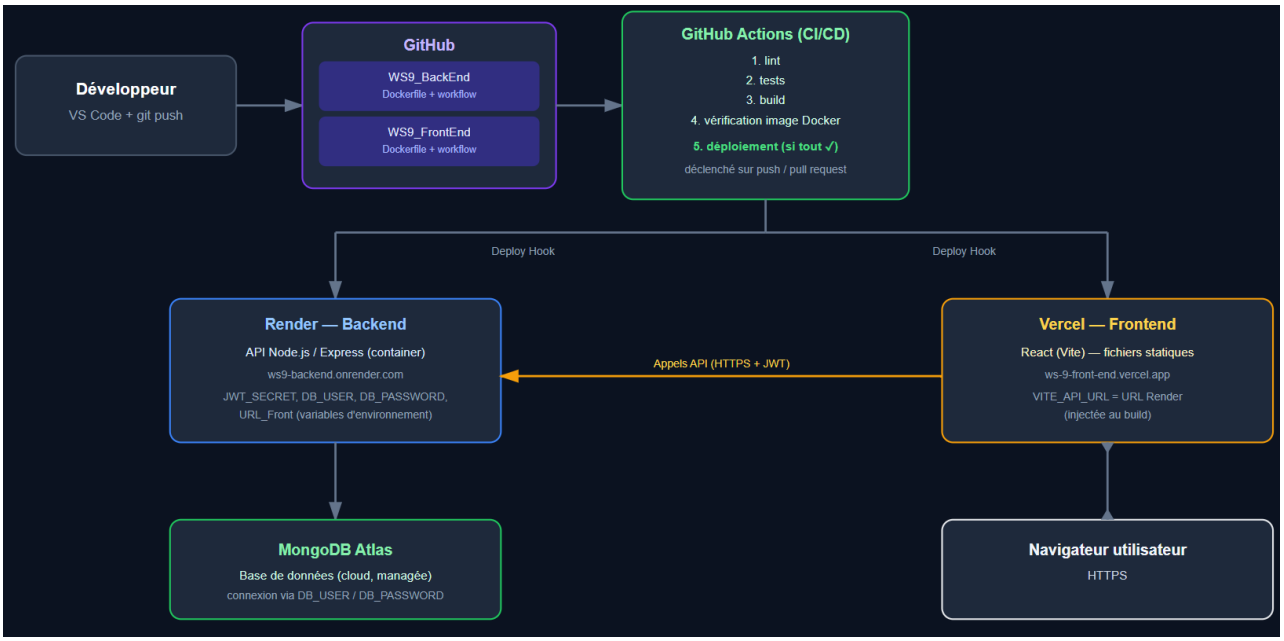
Pour lancer le déploiement, le pipeline appelle un Deploy Hook : une adresse fournie par Render et par Vercel qui déclenche une nouvelle mise en ligne. Cette adresse donne le droit de déployer l'application, je ne l'ai donc pas écrite dans le code. Je l'ai enregistrée dans les secrets de GitHub (RENDER_DEPLOY_HOOK et VERCEL_DEPLOY_HOOK), et le workflow la lit avec ${{ secrets.... }}. Elle n'apparaît ni dans le dépôt ni dans les logs.



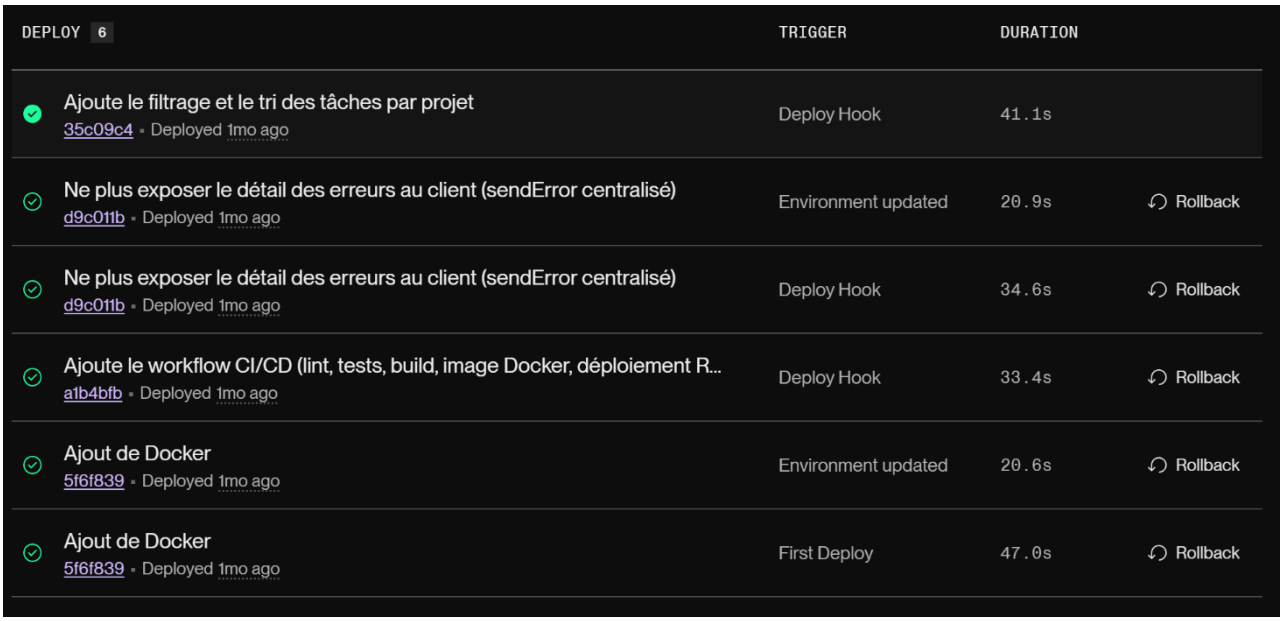
D. Hébergement et mise en production

J'ai choisi un hébergement adapté à chaque partie :
- le frontend sur Vercel, spécialisé dans les sites statiques ;
- le backend sur Render, qui fait tourner un serveur en continu à partir de mon image Docker ;
- la base de données sur MongoDB Atlas, une base managée dans le cloud.

En production, les variables d'environnement sont renseignées directement dans les paramètres de Render et de Vercel, et ne sont jamais dans le code.



Exemple d'un cycle complet : pour la fonctionnalité de filtrage et de tri des tâches, j'ai fait une modification sur le backend et une sur le frontend. Après chaque push sur main, GitHub Actions a lancé le pipeline. Toutes les étapes étant au vert, il a appelé le Deploy Hook, puis Render et Vercel ont mis en ligne la nouvelle version automatiquement. Sur le tableau de bord de Render, la colonne « Trigger » indique « Deploy Hook » : le déploiement a bien été déclenché par le pipeline.



En cas de problème, il est possible de revenir à une version précédente : avec le bouton « Instant Rollback » sur Vercel, ou en redéployant un ancien déploiement sur Render.

VII. Logiciels utilisés

Outils de développement
- Visual Studio Code : éditeur de code ;
- Git et Git Bash : gestion des versions et terminal ;
- GitHub : hébergement des dépôts et pipeline CI/CD (GitHub Actions) ;
- Postman : tests manuels des routes de l'API ;
- Docker Desktop : construction et lancement des conteneurs ;
- MongoDB Atlas : base de données dans le cloud ;
- Render et Vercel : hébergement du backend et du frontend.

Backend (Node.js)
- Express : création de l'API REST ;
- Mongoose et mongoose-unique-validator : modèles de données et unicité de l'email ;
- express-validator : validation des données reçues ;
- bcrypt : hachage des mots de passe ;
- jsonwebtoken : création et vérification des tokens JWT ;
- dotenv et cross-env : chargement des variables d'environnement selon le mode (dev ou prod) ;
- cors et express-rate-limit : filtrage de l'origine des requêtes et limitation du nombre de requêtes ;
- multer : gestion des fichiers images ;
- swagger-jsdoc et swagger-ui-express : documentation de l'API ;
- nodemon : redémarrage automatique du serveur en développement ;
- modules intégrés à Node.js : http (création du serveur) et dns (résolution des adresses).

Frontend
- React et React DOM : construction de l'interface ;
- React Router : navigation entre les pages ;
- Axios : requêtes vers l'API ;
- Vite : serveur de développement et build ;
- Tailwind CSS : mise en forme.

Tests et qualité
- Jest, Supertest et mongodb-memory-server : tests du backend ;
- Vitest, React Testing Library et jsdom : tests du frontend ;
- ESLint et Prettier : qualité et mise en forme du code.